VERIFICATION OF PROBING SECURITY FOR MASKED CIRCUITS: A CONSTRUCTIVE APPROACH AND THEORETICAL STUDY

by
KAĞAN GÜLSÜM

Submitted to the Graduate School of Engineering and Natural Sciences
in partial fulfilment of
the requirements for the degree of Master of Science
in Computer Science and Engineering

Sabancı University
July 2026

VERIFICATION OF PROBING SECURITY FOR MASKED CIRCUITS: A CONSTRUCTIVE APPROACH AND THEORETICAL STUDY

Approved by:

Asst. Prof. SÜHA ORHUN MUTLUERGİL .....
(Thesis Supervisor)

Prof. ERKAY SAVAŞ .....

Asst. Prof. KLAUS FREIHERR VON GLEISSENTHAL .....

Date of Approval: July 21, 2026

ABSTRACT

VERIFICATION OF PROBING SECURITY FOR MASKED CIRCUITS: A CONSTRUCTIVE APPROACH AND THEORETICAL STUDY

KAĞAN GÜLSÜM

Computer Science and Engineering, M.Sc. Thesis, July 2026

Thesis Supervisor: Asst. Prof. SÜHA ORHUN MUTLUERGİL

Keywords: masking, probing security, formal verification, side-channel analysis, coupling

Masking defends cryptographic hardware and software implementations against side-channel attacks by splitting each secret into randomised shares. The probing model of Ishai, Sahai, and Wagner reduces the security of a masked circuit to a combinatorial question: is every set of d wires, called a probe, jointly independent of the secrets? This thesis studies the exact verification of that property, namely d-probing security, for masked arithmetic circuits over the two-element field. We first identify the computational complexity of the problem. Computing the distribution of a single probe is #P-complete, and the corresponding security decision problem is complete for the exact-counting class $\mathrm{C}_{=}\mathrm{P}$, so verifying even a single probe sits above the polynomial hierarchy. Hence, we verify constructively and present Coppula, a d-probing security checker implemented in Lean 4 featuring a layered design. At its core is a rewrite engine acting as a probe oracle, which uses substitutions and a multiplicative factoring step to expose and isolate masks buried inside products. It operates alongside a subset-space traversal that avoids explicit enumeration of every d-subset by partitioning the space via a set-level Vandermonde identity. Ultimately, a successful discharge is interpreted as a measure-preserving coupling of the randomness space, extending the security guarantee of one certified probe to a larger safe set of wires. The checker is sound but necessarily incomplete; we formally prove its guarantees and evaluate it on masked gadgets, notably verifiying a family of cicruits where comparable tools fail.

ÖZET

MASKELENMIŞ DEVRELER IÇIN SONDALAMA GÜVENLIĞININ DOĞRULANMASI: YAPICI BIR YAKLAŞIM VE KURAMSAL BIR İNCELEME

KAĞAN GÜLSÜM

Bilgisayar Bilimi ve Mühendisliği, Yüksek Lisans Tezi, Temmuz 2026

Tez Danışmanı: Dr. Öğr. Üyesi SÜHA ORHUN MUTLUERGİL

Anahtar Kelimeler: maskeleme, sondalama güvenliği, biçimsel doğrulama, yan kanal analizi, eşlenme

Maskeleme, her bir sırrı rastgeleleştirilmiş paylara bölerek kriptografik donanım ve yazılım gerçekleştirimlerini yan kanal saldırılarına karşı korur. Ishai, Sahai ve Wagner'in sondalama modeli, maskelenmiş bir devrenin güvenliğini kombinatorik bir soruya indirger: *sonda* olarak adlandırılan her biri d kablodan oluşan kümeler, sırlardan bağımsız mıdır? Bu tez, iki elemanlı cisim üzerindeki maskelenmiş aritmetik devreler için bu özelliğin, yani d-sondalama güvenliğinin, tam doğrulamasını incelemektedir. Öncelikle problemin hesaplama karmaşıklığını belirliyoruz. Tek bir sondanın dağılımını hesaplamak #P-tamdır ve karşılık gelen güvenlik karar problemi, tamsayma sınıfı $\mathrm{C}_{=}\mathrm{P}$ için tamdır; dolayısıyla tek bir sondayı doğrulamak bile polinom hiyerarşisinin üzerinde yer alır. Bu nedenle doğrulamayı yapıcı bir biçimde gerçekleştiriyor ve Lean 4'te gerçekleştirilmiş, katmanlı bir tasarıma sahip bir d-sondalama güvenlik denetleyicisi olan Coppula'yı sunuyoruz. Çekirdeğinde, bir sonda kehaneti olarak işlev gören bir yeniden yazma motoru bulunur; bu motor, çarpımların içine gömülü maskeleri açığa çıkarmak ve izole etmek için ikameler ve çarpanlara ayırma adımı kullanır. Bu motor, her bir d-alt kümesinin açıkça numaralandırılmasından kaçınan ve uzayı bir küme-düzeyi Vandermonde özdeşliği aracılığıyla bölümleyen bir alt küme-uzayı gezinmesiyle birlikte çalışır. Nihayetinde, başarıyla kapatılan bir ispat yükümlülüğü, rastgelelik uzayının ölçü-koruyan bir eşlenmesi olarak yorumlanır ve bu da tek bir sertifikalı sondanın güvenlik garantisini daha büyük, güvenli bir kablo kümesine genişletir. Denetleyici sağlamdır ancak zorunlu olarak eksiktir;

garantilerini biçimsel olarak ispatlıyor ve maskelenmiş devrelerlar üzerinde, özellikle karşılaştırılabilir araçların başarısız olduğu bir devre ailesini doğrulayarak, değerlendiriyoruz.

ACKNOWLEDGEMENTS

I would like to thank my advisor and family.

Dedicated to my grandparents...

TABLE OF CONTENTS

ABSTRACT ..... **iv**
ÖZET ..... **v**
LIST OF TABLES ..... **xiii**
LIST OF FIGURES ..... **xiv**
LIST OF SYMBOLS ..... **xv**
LIST OF ABBREVIATIONS ..... **xvii**
1. INTRODUCTION **1**
1.1. Side-Channel Attacks and the Masking Countermeasure 1
1.2. From Physical Leakage to a Probing Adversary 2
1.3. Verifying Masked Implementations 3
1.4. Scope and Threat Model 3
1.5. Contributions 4
1.6. Thesis Outline 5
2. PRELIMINARIES **6**
2.1. Notation and Mathematical Background 6
2.1.1. The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials 6
2.1.2. Distributions, Laws, and Independence 7
2.1.3. Boolean Circuits and Gates 8
2.1.3.1. The Syntax of a Wire 9
2.1.3.2. The Semantics of a Wire 10
2.2. Secret Sharing and Masking 11
2.2.1. Boolean Masking 11
2.3. The ISW Probing Model 12
2.3.1. Threshold Probing and Observation Sets 12

2.3.2. d-Probing Security 13
2.4. Randomness Substitution 14
2.5. Counting Complexity Classes 16
3. RELATED WORK 18
3.1. Language-Based Verification 18
3.1.1. Verified Proofs of Higher-Order Masking 18
3.1.2. Large Sets and the Exploration Algorithm 19
3.1.3. Witness Replay and Derivation Reuse 19
3.1.4. MaskVerif and Physical Defaults 20
3.2. Fourier-Analytic Verification 20
3.2.1. The Xiao–Massey Criterion and Approximations 20
3.2.2. SAT-Based Realization and Its Cost 21
3.3. Other Approaches 21
3.3.1. Model Counting and SMT 21
3.3.2. Compositional Reasoning and Relabeling 22
3.4. Position of This Work 22
4. COUPLINGS AND PROBING SECURITY 23
4.1. The Complexity of Deciding Secret-Independence 23
4.1.1. From Independence to Distribution Counting 23
4.1.2. Counting the Law Is #P-Complete 24
4.1.3. Deciding Independence is $C_{=}$P-Complete 25
4.1.4. What the Placements Mean for Verification 26
4.2. Couplings of Probabilistic Programs 27
4.2.1. Coupling Two Runs Under Different Secrets 27
4.3. Probing Security via Couplings 28
4.3.1. d-Probing Security as a Family of Couplings 29
4.4. Substitutions as a Coupling Construction 29
4.4.1. The Derivation of an Explicit Coupling Witness 29
5. DESIGN OF COPPULA 32
5.1. Pipeline Overview 32
5.2. Component I: The Rewrite Engine 33
5.2.1. The Record T 34

5.2.2. The Simple Rewrite Rule 34
5.2.3. Fallback: General Substitution 36
5.2.4. The Factoring Step 37
5.2.5. Soundness of the Rules 38
5.2.6. Relation to the Coupling-Construction View 39
5.3. Component II: Traversing the Subset Space 40
5.3.1. The Representation of the Subset Space 40
5.3.2. Large-Set Pruning 41
5.3.3. Completeness via Vandermonde's Identity 42
5.4. Component III: Extending Probe Sets 44
5.4.1. Closure-Driven Extension 45
5.4.2. Replay-Driven Extension 46
5.4.3. Coupling-Driven Extension 47
6. IMPLEMENTATION OF COPPULA 50
6.1. Expression Representation 50
6.1.1. A Hash-Consed n-Ary Circuit DAG 51
6.1.2. Canonicalization: Flattening and Cancellation 52
6.1.3. Multiplicative Factoring 53
6.1.3.1. The Factoring Transformation 53
6.1.3.2. Enlarging the Class of Verifiable Circuits 54
6.2. Implementing the Rewrite Engine 55
6.2.1. Reference-Counted Single-Occurrence Discharge 55
6.2.2. Recomputing the Find-Rules per Round 56
6.3. Evaluating the Coupling 57
6.3.1. Bit-Sliced Evaluation Through a Coupled Environment 57
6.3.2. From Probabilistic Filter to Exact Verdict 58
7. EVALUATION 59
7.1. Benchmarks 59
7.2. Methodology and Metrics 60
7.3. Results 60
7.3.1. Verification Outcomes and Case Studies 61
7.3.2. Redundancy in the Search 62

7.3.3. The Cost and Benefit of Extending Probe Sets 62
7.4. Discussion 64
8. FUTURE WORK 66
8.1. Arithmetic Circuits on Larger Fields 66
8.2. Factoring Heuristics 67
8.3. Composing Couplings for Unions of Safe Sets 67
8.4. Symmetry-Orbit Space Exploration 68
8.5. Scaling and Compositional Verification 68
8.6. Glitches and Practical Aspects 69
9. CONCLUSION 70
BIBLIOGRAPHY 72
APPENDIX - ADDITIONAL EXPERIMENTAL DATA 75

LIST OF TABLES

Table 7.1. Extension-mechanism comparison across the benchmark suite 63

LIST OF FIGURES

Figure 7.1. Verification time versus circuit depth 61

LIST OF SYMBOLS

$\mathbb{F}_2$ The two-element field $\{0,1\}$.

$\oplus$ Addition in $\mathbb{F}_2$ (exclusive-or).

$\odot$ Multiplication in $\mathbb{F}_2$ (logical-and).

$\uplus$ Union of sets asserted to be disjoint; $\biguplus$ is its indexed form.

$\mathcal{S}$ Set of secret input wires of a circuit.

$\mathcal{R}$ Set of random (masking) input wires of a circuit.

$\mathcal{P}$ Set of public input wires of a circuit.

$\mathcal{V}$ The input wires of a circuit, $\mathcal{V} = \mathcal{S} \uplus \mathcal{R} \uplus \mathcal{P}$.

$\mathcal{W}$ Set of all wires (vertices) of a circuit.

$\mathrm{expr}(w)$ The $\mathbb{F}_2$ polynomial in the input variables obtained by reading the circuit graph at w.

$[\![\cdot]\!]$ Denotation of an expression.

$\mathrm{fv}(e)$ The input variables an expression e mentions.

$\mathrm{vars}(w)$ The random inputs a wire w mentions, $\mathrm{vars}(w) = \mathrm{fv}(w) \cap \mathcal{R}$.

ρ A randomness assignment, or tape, $\rho : \mathcal{R} \to \{0,1\}$.

$\mathbf{a}, \mathbf{b}$ Secret assignments.

$\mathbf{w}$ A probe, which is a tuple $\mathbf{w} = (w_1, \ldots, w_k)$ of at most d wires.

o An observation $o \in \{0,1\}^k$ that a probe can take.

$N_{\mathbf{a}}(o)$ The number of tapes ρ on which a probe reads the observation o under secret assignment $\mathbf{a}$.

$\mathcal{L}_{\mathbf{a}}(\mathbf{w})$ The law of a probe $\mathbf{w}$ under secret assignment $\mathbf{a}$, taken over uniform ρ; $\mathcal{L}(Z)$ denotes the law of a random variable Z in general.

d Probing order (maximum number of wires an adversary may observe).

σ Randomness substitution $r \mapsto e \oplus r$.

$\widehat{\sigma}$ The bijection of the randomness space induced by σ.

$\varphi_{\mathbf{a}}$ The composite tape map under a single secret assignment $\mathbf{a}$.

$\mathbf{\Phi}$ A family of bijections of the random space.

P The class of decision problems solvable in deterministic polynomial time.

NP The class of decision problems solvable in nondeterministic polynomial time.

coNP The class of decision problems whose complements lie in NP.

PH The polynomial hierarchy.

#P The class of functions that count the accepting computations of a nondeterministic polynomial-time machine.

$\mathrm{C}_{=}\mathrm{P}$ The exact-counting complexity class.

GapP The closure of #P under subtraction.

$\mathrm{P}^{\#\mathrm{P}}$ The class of decision problems solvable in polynomial time given an oracle for a #P counting function.

LIST OF ABBREVIATIONS

ANF Algebraic Normal Form.

DAG Directed Acyclic Graph.

ISW Ishai, Sahai, Wagner.

SAT Boolean Satisfiability.

ExactSAT Exact Boolean Satisfiability.

#SAT Sharp Boolean Satisfiability.

SMT Satisfiability Modulo Theories.

XOR Exclusive-OR.

1. INTRODUCTION

1.1 Side-Channel Attacks and the Masking Countermeasure

A cryptographic scheme can be secure on paper and insecure on the bench. When an algorithm runs on a real device its power draw, its timing, and its electromagnetic emanations all vary with the data it handles, and an adversary who measures those physical quantities can recover a key the mathematics was supposed to protect. This is called a *side-channel attack*, and differential power analysis (Kocher et al., 1999) is a canonical example. By correlating many power traces against a guess of an intermediate value, it extracts secret keys from implementations that are deemed secure in theory. So, the insecurity lies not in the cipher but in the way the hardware computes it.

Masking is the standard algorithmic countermeasure against such attacks (Chari et al., 1999). It splits each secret value into n *shares* so that any $n-1$ of them, taken together, are independent of the secret. Then, computations are carried out on the shares rather than on the secret itself. The shares are recombined by an operation fixed in advance, and the choice of operation names the scheme: arithmetic masking adds the shares in a ring and multiplicative masking multiplies them, but the form we adopt is *Boolean masking*, where the operation is exclusive-or over $\mathbb{F}_2$. A secret x is written as $x = x_1 \oplus \cdots \oplus x_n$ with the first $n-1$ shares drawn uniformly at random, so that any $n-1$ of them reveal nothing about x.

The reason this helps countering side-channel attacks is physical. Each share leaks separately and noisily, and a measurement that reflects only some of the shares of a secret is, by construction, independent of that secret; it carries no information an attacker can use. To learn anything the attacker must combine the leakage of several shares at once and average away the noise, and the cost of doing so grows

steeply. For a fixed noise level, the number of traces needed to execute a successful attack grows exponentially in the number of shares (Chari et al., 1999). Adding a share is therefore a tunable defence as well, trading circuit size for attack cost.

The difficulty is that this gurantee holds only if the shares are kept apart at places the attacker can reach. Implementing masking across a whole cipher is delicate, because the non-linear operations of the unmasked algorithm force the shares of different variables to interact in the masked version, and it is precisely those interactions that a scheme must handle with care (Ishai et al., 2003; Rivain & Prouff, 2010). Simple mistakes can reopen the leak, which masking was meant to close.

1.2 From Physical Leakage to a Probing Adversary

Reasoning about masking directly in terms of noisy power traces is unwieldy. The adversary's advantage is a statistical quantity that depends on the device, the measurement setup, and the noise distribution, none of which the designers control completely. The field instead reasons about masking through a discrete abstraction, called the *probing model* owing to Ishai, Sahai, and Wagner (Ishai et al., 2003). In this model an adversary of order d does not measure anything; it simply chooses any d wires of the circuit and is handed their exact values. The circuit is *d-probing secure* when no such choice of d wires lets the adversary tell one secret from another. An order-d masking accordingly uses $d+1$ shares, so that no d probes ever meet all the shares of a secret at once.

The probing model trades physical fidelity for a question one can actually decide. Security becomes a combinatorial property of the circuit, checkable in principle by inspecting finitely many sets of wires. What makes the trade legitimate is that the abstraction is not merely convenient but provably faithful. Duc, Dziembowski, and Faust (Duc et al., 2014) gave a reduction showing that security in the probing model implies security in the more realistic *noisy leakage* model, in which every wire leaks a noisy function of its value; the abstract, value-based guarantee thus entails the physical one. We should note that the reduction is not trivial, it asks that the noise grow with the number of shares, and it carries a dependence on the field size, but establishing it as a question about leakage models rather than about any particular circuit, is useful in practice. We therefore take d-probing security as our target throughout and do not revisit this bridge. The reader should simply keep in mind

that a d-probing-secure circuit is what a faithful masked implementation is supposed to achieve.

1.3 Verifying Masked Implementations

Masking is easy to get subtly wrong, and the probing model makes the failures precise. A single wire may recombine several shares of one secret, or a random mask meant to hide one value may cancel against a copy of itself elsewhere; either case reopens the very leak the masking was meant to close. Correctness cannot be judged wire by wire as well, because d-probing security is a statement about how wires leak *jointly*, and the number of d-subsets grows quickly with the size of the circuit and the order d, which puts checking by hand out of reach for all but the smallest implementations.

What one wants, then, is a tool that takes a masked circuit and decides, mechanically and for all d-subsets at once, whether any of them leaks. This is the problem of *formally verifying* a masked implementation, and a line of work has grown around it (Barthe et al., 2015, 2019). Two difficulties stand in the way. Even a single subset already poses a question about a distribution over all of the circuit's internal random choices and there are combinatorially many subsets to settle. The tools that scale do not compute those distributions explicitly.

1.4 Scope and Threat Model

Before moving on, we should fix the model precisely. The circuit definition we work with is arithmetic circuits over $\mathbb{F}_2$, built from two-input XOR and AND gates, and its inputs are partitioned into secret, random, and public wires. Leakage is *value-based*, so, a probed wire leaks the logical value it carries and nothing more. Physical effects that make a wire leak more than its final value — such as glitches and transitions in hardware (Barthe et al., 2019) — are outside the model, as is any question of composing separately verified circuits into a larger one. The object of study is the exact probing security of a single circuit, decided over its wire values.

Within that model our aim is exact verification rather than a sound approximation. So, when the tool reports a circuit secure, every one of its $\sum_{i \le d} \binom{|\mathcal{W}|}{i}$ probes has been accounted for; when it reports a circuit insecure it returns a specific leaking probe as a witness — with the exception of the incomplete fragment flagged in Chapter 5.

1.5 Contributions

This thesis presents Coppula[1], a verifier of d-probing security for masked arithmetic circuits over $\mathbb{F}_2$, implemented in Lean 4. Its design rests on three ideas. Before developing them we place the verification problem itself in the complexity hierarchy, and the contributions follow in that order.

Placing the exact-verification problem. Before building anything we ask how hard the problem really is. The law of a probe is actually a histogram of integer tape counts, and we show that computing one such count is #P-complete, while deciding whether two secrets leave the law unchanged compares two such counts and is $\mathrm{C}_{=}$P-complete (Chapter 4). Therefore, even a single probe lands on a level of the complexity hierarchy above the polynomial hierarchy, which is what rules out exact counting and motivates the constructive route the rest of our tool takes.

Deciding one probe by construction. At the heart of the tool is a rewrite engine that decides whether a single probe depends on the secret by substituting its randoms away one at a time, driving the probe toward a secret-free form (Chapter 5). The engine is layered as a cheap single-occurrence rule first, followed by a complete substitution loop behind it, and we prove its verdicts sound. Some masks, though, are buried inside products where ordinary substitution cannot reach them, and to expose those we cleverly add *multiplicative factoring* steps that reshape products so the hidden randoms surface.

Covering every probe without enumerating them. Certifying each d-subset in turn is infeasible, so the tool instead splits the space of probes recursively, in the manner of `maskVerif` (Barthe et al., 2019), and tries to discharge whole regions at once. The delicate point, of course, is that the split must lose nothing. So, we prove it covers every probe exactly once, by reading a Vandermonde-style subset-counting identity at the level of sets (Chapter 5).

1. https://github.com/Sabanci-Theory/coppula/

Reading a proof as a coupling, and growing the probe. A successful discharge can be read as more than a yes-or-no verdict. We reinterpret it as a measure-preserving *coupling* of the circuit's randomness, in the tradition of coupling proofs for probabilistic programs (Barthe et al., 2017), and develop this reading for probing security in Chapter 4. The benefit is that one certificate can apply for many wires, in turn we grow a certified probe into a whole *safe set* of wires that a single discharge settles. We study three mechanisms for this growth, and adopt the coupling-based one, proving that the other two sit inside it (Chapter 5).

Implementation and evaluation. We describe the data structures that make the engine efficient (a hash-consed circuit representation and a bit-sliced test for the extension) (Chapter 6). and we evaluate Coppula on a suite of masked gadgets, reporting its verification outcomes (Chapter 7).

1.6 Thesis Outline

Chapter 2 fixes notation and defines arithmetic circuits, the probing model, and d-probing security, and studies the randomness-substitution technique we rely on throughout. Chapter 3 surveys the verification literature we build on and depart from. Chapter 4 develops the coupling view of probing security and situates the underlying decision problem in the complexity hierarchy. Chapter 5, the core of the thesis, presents the algorithm with its three components together with their correctness proofs, and Chapter 6 gives the concrete data structures behind them. Chapter 7 reports the experiments. Chapter 8 outlines the directions our approach does not treat, and Chapter 9 concludes the work.

2. PRELIMINARIES

2.1 Notation and Mathematical Background

This chapter collects the background the rest of the thesis builds on and fixes the notation for it. We use $\{0,1\}$ or $\mathbb{F}_2$ for the set of bits. Tuples are set in boldface, $\mathbf{x} = (x_1, \ldots, x_n)$, with $|\mathbf{x}|$ denoting their length. For a finite set X we write $\mathcal{P}(X)$ for its power set and $\binom{X}{k}$ for the family of its k-element subsets. We write $X \uplus Y$ for $X \cup Y$ when the two sets are disjoint, and $\biguplus$ for the indexed form. Functions are total unless we say otherwise, and we identify a function $\{0,1\}^n \to \{0,1\}$ with the Boolean function it computes. Notation that is used only locally is introduced where it appears.

2.1.1 The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials

The field $\mathbb{F}_2$ has carrier $\{0,1\}$ with addition $\oplus$, the exclusive-or, and multiplication $\odot$, the logical-and. Every element is its own additive inverse, so $x \oplus x = 0$, and multiplication is idempotent, $x \odot x = x$.

Every function $f : \mathbb{F}_2^n \to \mathbb{F}_2$ is a polynomial over $\mathbb{F}_2$, and idempotence lets us take that polynomial to have no variable raised above the first power. So f has a unique *multilinear* representation,

$$f(x_1, \ldots, x_n) \;=\; \bigoplus_{S \subseteq [n]} c_S \bigodot_{i \in S} x_i, \qquad c_S \in \mathbb{F}_2,$$

its *algebraic normal form* (ANF). Uniqueness is what turns the ANF into a decision

procedure for the questions we put to it. In particular, note that, f depends on a variable x_i (meaning that flipping x_i can change the output) if and only if some monomial of the ANF contains x_i.

2.1.2 Distributions, Laws, and Independence

Probing security is a statement about distributions, so we fix the little background we need here.

A *distribution* on a finite set X is a function $\mu : X \to [0,1]$ with $\sum_{x\in X} \mu(x) = 1$. It extends to subsets by summing, $\mu(A) = \sum_{x\in A} \mu(x)$ for $A \subseteq X$, and in that form it is called a *measure*, so a distribution is a measure of total weight 1, and we use the two words interchangeably. The *uniform* distribution assigns $\mu(x) = 1/|X|$ to every point, hence $\mu(A) = |A|/|X|$ to every subset. On a cube $X = \{0,1\}^n$ the uniform distribution is the *product* of the uniform distribution on each of the n coordinates, giving $\mu(x) = 2^{-n}$ for every $x \in X$.

A random variable Z into a finite set X induces the distribution $x \mapsto \Pr[Z = x]$, its *law*, which we write $\mathcal{L}(Z)$. Two random variables are *identically distributed* when their laws agree on every value.

The security notion we are after is the independence of a family of laws from a parameter, which we state below.

Definition 2.1 (Independence of a parameter)**.** *Let $(Z_s)_{s\in\Sigma}$ be a family of random variables into X, one for each value of a parameter s, and write $\mu_s = \mathcal{L}(Z_s)$ for their laws, so that s selects one distribution out of the family $(\mu_s)_{s\in\Sigma}$. The family is* independent of s *when*

$$\mu_s \;=\; \mu_{s'} \qquad \textit{for all } s, s' \in \Sigma.$$

We prove that two laws are equal by carrying one onto the other with a map that does not disturb the distribution.

Definition 2.2 (Measure-preserving map)**.** *A map $\pi : X \to X$ on a finite set is* measure-preserving *for a distribution μ when*

$$\mu\Big(\pi^{-1}(A)\Big) \;=\; \mu(A) \qquad \textit{for every } A \subseteq X.$$

If Z has law μ, then $\pi(Z)$, the random variable obtained by applying π to each outcome, has law $A \mapsto \mu(\pi^{-1}(A))$, which is called the *pushforward* of μ along π. The condition in the above definition says exactly that the pushforward is μ again.

For the uniform distribution the condition becomes bijectivity. As $|\pi^{-1}(A)| = |A|$ for every A forces π to be injective, and an injection of a finite set into itself is onto. Every measure-preserving map in this thesis is of this kind, so the phrase may be read as "bijective" throughout. The technique we use simply states that if π preserves the uniform distribution on X and Z is uniform, then for any function g the variables $g(Z)$ and $g(\pi(Z))$ are identically distributed.

2.1.3 Boolean Circuits and Gates

We now introduce the object the rest of the thesis is built on. The definition is the standard digraph formulation of a circuit, specialised to arithmetic over the two-element field. On top of that graph we then read off, for each wire, the $\mathbb{F}_2$ polynomial it computes. So, the graph is the circuit, the polynomial on the wire is the *syntax* our checker rewrites, and the function that polynomial denotes is the *semantics* every claim in the thesis is ultimately about.

Definition 2.3 (Arithmetic circuit over $\mathbb{F}_2$)**.** *Fix the field $\mathbb{F}_2 = \{0,1\}$ and a set $S = \{s_i\}$ of* gate functions*, each of some arity $a_i \in \mathbb{N}_{\geq 1}$, i.e. each s_i maps $\mathbb{F}_2^{a_i}$ to $\mathbb{F}_2$. A* circuit *is a triple $C = (\mathcal{W}, \lambda, \mathrm{op})$ in which*

- *$\mathcal{W}$ is a finite set whose elements we call the* wires*, the vertices of the graph, partitioned as*

$$\mathcal{W} \;=\; \underbrace{\mathcal{S} \uplus \mathcal{R} \uplus \mathcal{P}}_{\text{input wires } \mathcal{V}} \uplus\, \mathcal{G}$$

  into secret*,* random *and* public *inputs (jointly constituting the input wires $\mathcal{V}$, and each carrying a variable) together with the* computation gates *$\mathcal{G}$,*

- *λ labels each computation gate $w \in \mathcal{G}$ with a gate function $\lambda(w) \in S$,*

- op *assigns to each computation gate $w \in \mathcal{G}$ its* operand tuple *$\mathrm{op}(w) = (u_1, \ldots, u_a) \in \mathcal{W}^a$, where a is the arity of $\lambda(w)$.*

subject to the requirement that the digraph $(\mathcal{W}, E)$ with edge set

$$E \;=\; \Big\{ (u,w) \mid w \in \mathcal{G},\; u \in \mathrm{op}(w) \Big\}$$

contain no directed cycle. The input wires $\mathcal{V}$ *have indegree* 0*; a computation gate has indegree the arity of its label, and a vertex of outdegree* 0 *is an* output gate*. For a computation gate we speak interchangeably of the gate and the wire carrying its output.*

Over $\mathbb{F}_2$ there is no difference between the Boolean and the arithmetic view of such a circuit: exclusive-or is addition, logical-and is multiplication, and the two together already express every function $\mathbb{F}_2^n \to \mathbb{F}_2$, i.e. they are universal. So we fix

$$S = \{\oplus,\ \odot\},$$

addition and multiplication in $\mathbb{F}_2$ (Section 2.1.1), both of arity 2. Every computation gate then has indegree exactly 2, with $\mathrm{op}(w_i) = (w_j, w_k)$, and reads

$$w_i = w_j \oplus w_k \qquad \text{or} \qquad w_i = w_j \odot w_k.$$

Throughout the thesis $\odot$ denotes multiplication in $\mathbb{F}_2$; we might sometimes use $\cdot$ for the same operation.

Acyclicity is what makes a circuit computable. It lets us list the wires in a topological order $w_1, \ldots, w_m$ in which the two operands of every gate appear before the gate itself, so that $j, k < i$.

2.1.3.1 The Syntax of a Wire

Reading the graph along that order turns each wire into a polynomial over $\mathbb{F}_2$ in the input variables. The polynomials in question are the *expressions*

$$e \ ::= \ 0 \mid 1 \mid v \mid e_1 \oplus e_2 \mid e_1 \odot e_2, \qquad v \in \mathcal{V} = \mathcal{S} \uplus \mathcal{R} \uplus \mathcal{P},$$

written as they are built in the graph.

Definition 2.4 (The expression of a wire)**.** *For* $w \in \mathcal{W}$ *the expression* $\mathrm{expr}(w)$ *is defined by recursion along the topological order:*

$$\mathrm{expr}(w) = \begin{cases} v & \text{if } w \in \mathcal{V} \text{ denotes the variable } v, \\ \mathrm{expr}(w_j) \star \mathrm{expr}(w_k) & \text{if } w \in \mathcal{G} \text{ and } \mathrm{op}(w) = (w_j, w_k),\ \star = \lambda(w). \end{cases}$$

The recursion is well founded exactly because the graph is acyclic. So, the operands of w_i are w_j, w_k with $j, k < i$, hence the second clause always descends. What it produces mentions no wire and only the inputs. The sharing present in the graph is unfolded away.

2.1.3.2 The Semantics of a Wire

Take as an *input assignment* a map $\eta : \mathcal{V} \to \mathbb{F}_2$. Since $\mathcal{V}$ is a disjoint union, we write it as a triple $\eta = (\mathbf{a}, \rho, \pi)$ with a secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$, a *tape* $\rho \in \{0,1\}^{\mathcal{R}}$, and a public assignment $\pi \in \{0,1\}^{\mathcal{P}}$. The expression of a wire denotes a bit value under such an assignment, by structural recursion on the grammar:

$$[\![0]\!]\eta = 0, \qquad [\![1]\!]\eta = 1, \qquad [\![v]\!]\eta = \eta(v),$$

$$[\![e_1 \oplus e_2]\!]\eta = [\![e_1]\!]\eta \oplus [\![e_2]\!]\eta, \qquad [\![e_1 \odot e_2]\!]\eta = [\![e_1]\!]\eta \odot [\![e_2]\!]\eta,$$

where the two operators on the left of each of the last two clauses are syntax and the ones on the right are the field operations of Section 2.1.1. This is the usual reading of a polynomial as the function it computes.

Definition 2.5 (The wire function)**.** *The* wire function *of $w \in \mathcal{W}$ in the circuit C is*

$$[\![w]\!]_C \;=\; [\![\mathrm{expr}(w)]\!] \;:\; \{0,1\}^{\mathcal{S}} \times \{0,1\}^{\mathcal{R}} \times \{0,1\}^{\mathcal{P}} \;\longrightarrow\; \mathbb{F}_2.$$

So a wire *is*, semantically a Boolean function, and equivalently, by Section 2.1.1, a multilinear $\mathbb{F}_2$ polynomial function, which eats a secret, a random and a public assignment and returns one bit. The wire functions are built on top of one another over the shared variables of $\mathcal{V}$, since a gate's function is its two operands' functions combined by $\oplus$ or $\odot$, with the input wires supplying the variables everything else is built from. Fixing η and evaluating gate by gate along the topological order computes exactly $[\![w]\!]_C(\eta)$ for every w, so the polynomial reading and the operational one are essentially the same.

For example, a gate $c = p \oplus r$ whose operand p is itself the gate $s \odot m$ has $\mathrm{expr}(c) = (s \odot m) \oplus r$, a polynomial in the secret s and the randoms m, r, denoting the function $(\mathbf{a}, \rho) \mapsto \big(\mathbf{a}(s) \odot \rho(m)\big) \oplus \rho(r)$.

Since the publics are held fixed throughout the thesis we suppress π, and we follow the usual abuse of notation to denote with w both for the wire and for the function it

denotes (i.e., $w(\rho;\mathbf{a}) = [\![w]\!]_C(\mathbf{a},\rho,\pi)$). When the secrets are fixed as well we simply write $w(\rho)$. And a tuple of wires is made up componentwise.

Notation 2.1. *The* free variables $\mathrm{fv}(e) \subseteq \mathcal{V}$ *of an expression are those it mentions, defined by structural recursion:* $\mathrm{fv}(0) = \mathrm{fv}(1) = \emptyset$, $\mathrm{fv}(v) = \{v\}$, *and* $\mathrm{fv}(e_1 \star e_2) = \mathrm{fv}(e_1) \cup \mathrm{fv}(e_2)$. *For a wire we write* $\mathrm{fv}(w) = \mathrm{fv}(\mathrm{expr}(w))$, *so that* $\mathrm{fv}(w) = \{w\}$ *for an input wire and* $\mathrm{fv}(w_j) \cup \mathrm{fv}(w_k)$ *for a gate with operands* w_j, w_k. *We set*

$$\mathrm{vars}(e) \;=\; \mathrm{fv}(e) \cap \mathcal{R} \;\subseteq\; \mathcal{R}$$

for the random inputs e *mentions, extended to a tuple of wires trivially by* $\mathrm{vars}(\mathbf{w}) = \bigcup_i \mathrm{vars}(w_i)$, *and we call* e secret-free *when* $\mathrm{fv}(e) \cap \mathcal{S} = \emptyset$.

2.2 Secret Sharing and Masking

Physical implementations of cryptographic hardware can leak information. Power draw, electromagnetic emanation, and timing all correlate with the values a device computes on, and an attacker who measures them can recover a key that is out of cryptographic reach (Kocher et al., 1999). *Masking* is the standard algorithmic countermeasure (Chari et al., 1999). Instead of computing on a secret directly, one splits it into several *shares* whose joint distribution hides it, then rewrites the computation to run on the shares throughout and reconstruct only at the very end. The promise is that an attacker who sees few enough intermediate values learns nothing, because too few shares carry no information about the secret.

2.2.1 Boolean Masking

We use *Boolean* masking, where shares combine by exclusive-or. An *n-sharing* of a bit x is a tuple $(x_0, \ldots, x_{n-1})$ with $x_0 \oplus \cdots \oplus x_{n-1} = x$. To draw one, sample $x_1, \ldots, x_{n-1}$ uniformly and independently and set $x_0 = x \oplus x_1 \oplus \cdots \oplus x_{n-1}$.

Placing this in the circuit of Definition 2.3 fixes each variable's role. The shared bit is a secret input, $x \in \mathcal{S}$. The $n-1$ freshly sampled shares are random inputs, $x_1, \ldots, x_{n-1} \in \mathcal{R}$, and are exactly the coordinates a tape ρ assigns. The remaining

share x_0 is not an input but a computation gate, the $\oplus$ of x with the others, so $x_0 \in \mathcal{G}$ with $\mathrm{fv}(x_0) = \{x, x_1, \ldots, x_{n-1}\}$ and $\mathrm{vars}(x_0) = \{x_1, \ldots, x_{n-1}\}$.

Leaving out any one share destroys all information about x. Drop x_j for any j and let Z be the tuple of the remaining $n-1$ shares, indicating a random variable. Whatever the value of x, that tuple is uniform on $\mathbb{F}_2^{n-1}$, so the law of Z is the same for $x = 0$ and for $x = 1$. That is, it is independent of x in the sense of Definition 2.1. The full tuple is not, since its sum is x itself.

A masked circuit carries every value in shared form and replaces each operation by a *gadget*, a sub-circuit computing a sharing of the result from sharings of its inputs. Linear operations are trivial, since $\oplus$ acts share by share. Multiplication is where the difficulty and the fresh randomness live, such a gadget must combine shares of both inputs without ever forming a value that depends on a secret. The masked AND of Ishai, Sahai and Wagner (Ishai et al., 2003) is the archetype, and the circuits we verify are built from gadgets of this kind.

2.3 The ISW Probing Model

For the security notion to be precise we need to state an adversary. Ishai, Sahai and Wagner (Ishai et al., 2003) gave the model the thesis works in: the probing model. An adversary places a bounded number of probes on the wires of a circuit and reads the values they carry, and the circuit is secure when no choice of probes tells the adversary anything about the secrets. The model abstracts a physical side-channel attacker into a clean combinatorial game, and it is the setting in which masking admits proofs.

The connections between the physical side-channel attack and the probing model, however, are not exposited in this thesis.

2.3.1 Threshold Probing and Observation Sets

Fix a circuit with secret inputs $\mathcal{S}$, random inputs $\mathcal{R}$, and public inputs $\mathcal{P}$ as in Section 2.1.3. An adversary of order d selects at most d wires and learns their joint

values. That selection is the object the whole thesis reasons about, so we name it and give its notation.

Definition 2.6 (Probe)**.** *A* probe *of order d is a tuple $\mathbf{w} = (w_1, \ldots, w_k)$ of $k \leq d$ distinct wires of the circuit. Under a secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$ and a tape $\rho \in \{0,1\}^{\mathcal{R}}$ it takes the joint value*

$$\mathbf{w}(\rho; \mathbf{a}) \;=\; \Big(w_1(\rho; \mathbf{a}), \ldots, w_k(\rho; \mathbf{a})\Big) \;\in\; \{0,1\}^k,$$

each component being the wire function of Definition 2.5 with the publics held fixed. We write $\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}\Big(\mathbf{w}(\rho; \mathbf{a})\Big)$ for the law of the probe under $\mathbf{a}$, the distribution of $\mathbf{w}(\rho; \mathbf{a})$ over a uniform pick of ρ.

We follow the convention of Section 2.1 in setting the tuple in boldface, and we use $\mathbf{w}$ for a probe throughout the thesis. The literature also calls a probe an *observation set*, and calls a value $o \in \{0,1\}^k$ that the probe can take an *observation*. We use the word observation as well, and say "probe" for the tuple.

2.3.2 d-Probing Security

Definition 2.7 (d-probing security)**.** *A circuit is* d-probing secure *if every probe $\mathbf{w}$ of order d is* secret-independent*. So, its law, taken over uniform $\rho \in \{0,1\}^{\mathcal{R}}$, is independent of the secret assignment in the sense of Definition 2.1,*

$$\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}_{\mathbf{b}}(\mathbf{w}) \qquad \text{for all } \mathbf{a}, \mathbf{b} \in \{0,1\}^{\mathcal{S}}.$$

The order d counts the wires the adversary is granted, and an n-share masking is usually meant to withstand $d = n - 1$ of them. The definition quantifies over tuples of wires read from a single run of the circuit, rather than single indivdual wires, since a masking leaks only jointly. And it is a worst-case statement over all $\sum_{i \leq d} \binom{|\mathcal{W}|}{i}$ probes on the circuit's wires $\mathcal{W}$. Verifying this definition for the circuits we defined is our main task.

2.4 Randomness Substitution

This section isolates, as a single reusable fact, the probabilistic argument our checker automates. Fix a circuit as in Section 2.1.3, with random inputs $\mathcal{R}$, and recall that a *randomness assignment*, or *tape*, is a function $\rho : \mathcal{R} \to \{0,1\}$. The secrets and public inputs are held fixed throughout, so every wire w denotes a function $w(\rho)$ of ρ alone (Definition 2.5). A probe $\mathbf{w}$ is as in Definition 2.6, a tuple of at most d wires whose order does not affect the joint distribution, and Definition 2.7 asks that this joint distribution, taken over the uniformly distributed ρ, not move as the secrets vary.

The whole argument rests on one line of $\mathbb{F}_2$ arithmetic: for any fixed bit c, the translation/shear $b \mapsto c \oplus b$ is a *bijection* of $\{0,1\}$ that preserves the uniform distribution (i.e., it is its own inverse and merely swaps the two outcomes). Everything below is this fact, applied to a single coordinate of ρ with every other coordinate fixed.

Substitution is the syntactic half of that fact, and we mention that first. For $r \in \mathcal{R}$ and an expression t, write $r \mapsto t$ for the *substitution* sending r to t, and $e[r \mapsto t]$ for its application to an expression e, which replaces every occurrence of r in e by t. The brackets mark the application and the arrow names the substitution being applied. Application is by structural recursion on the grammar of Section 2.1.3:

$$r[r \mapsto t] = t, \qquad u[r \mapsto t] = u \quad (u \in \mathcal{V} \cup \{0,1\},\ u \neq r),$$

$$(e_1 \star e_2)[r \mapsto t] = e_1[r \mapsto t] \star e_2[r \mapsto t] \qquad (\star \in \{\oplus, \odot\}).$$

The grammar has no binders, so the recursion needs no side conditions.

Definition 2.8 (Randomness substitution)**.** *Let $r \in \mathcal{R}$ and let e be an expression with $r \notin \mathrm{vars}(e)$. The* substitution $\sigma = (r \mapsto e \oplus r)$ *acts on any expression e' by $e'\sigma = e'[r \mapsto e \oplus r]$, hence on a wire w through its expression,*

$$w\sigma \;=\; \mathrm{expr}(w)\,[r \mapsto e \oplus r],$$

and on a probe componentwise. A chain of substitutions is applied in turn, each to the expression the previous one produced. The same σ acts on randomness assignments, through the map $\widehat{\sigma} : \{0,1\}^{\mathcal{R}} \to \{0,1\}^{\mathcal{R}}$ whose image $\rho' = \widehat{\sigma}(\rho)$ shifts the r-coordinate by $e(\rho)$ and leaves every other coordinate untouched:

$$\rho'(r) = e(\rho) \oplus \rho(r), \qquad \rho'(r') = \rho(r') \quad \textit{for } r' \neq r.$$

Geometrically, $\widehat{\sigma}$ is a *shear* of the cube $\{0,1\}^{\mathcal{R}}$, simply, it fixes every coordinate but r and offsets that one by $e(\rho)$. On the r-coordinate alone it is the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$, which merely swaps the two outcomes when $e(\rho) = 1$ and does nothing when $e(\rho) = 0$, so it leaves that coordinate uniform. We call any such map, one coordinate translated by a context depending on the others, and everything else left intact, a *shear*, and use the word throughout.

We pair the two actions under one name with the lemma below. The syntactic rewrite $w \mapsto w\sigma$ and the reparametrisation $\rho \mapsto \widehat{\sigma}(\rho)$ are one operation seen from two sides, and $\widehat{\sigma}$ does not disturb the uniform distribution.

Lemma 2.1 (Substitution)**.** *For a substitution $\sigma = (r \mapsto e \oplus r)$ as in Definition 2.8, the map $\widehat{\sigma}$ is an involution of $\{0,1\}^{\mathcal{R}}$ preserving the uniform distribution (Definition 2.2), and for every wire w,*

$$(w\sigma)(\rho) \;=\; w\Big(\widehat{\sigma}(\rho)\Big) \qquad \textit{for every randomness assignment } \rho.$$

Proof. Fix every coordinate of ρ other than r. Since $r \notin \mathrm{vars}(e)$, no leaf of e is r, so evaluating e never reads the r-coordinate and $e(\rho)$ is a function of these fixed coordinates alone; it is therefore constant as $\rho(r)$ ranges over $\{0,1\}$. On this fibre $\widehat{\sigma}$ acts by the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$, which is its own inverse, so $\widehat{\sigma}$ is an involution and in particular a bijection of $\{0,1\}^{\mathcal{R}}$ — hence measure-preserving for the uniform distribution. Concretely, it leaves each coordinate uniform, and the uniform distribution on the cube is the product of those, so the cube stays uniform.

For the stated identity, replace w by its expression $\mathrm{expr}(w)$ (Definition 2.4), which mentions only input variables, and argue by structural induction on it. For a constant both sides are trivially equal. For $\mathrm{expr}(w) = r$ both sides equal $e(\rho) \oplus \rho(r)$, on the left by the clause $r[r \mapsto e \oplus r] = e \oplus r$, and on the right by the definition of $\widehat{\sigma}$. For an input variable $u \neq r$ both sides equal $u(\rho)$, since substitution leaves u alone and $\widehat{\sigma}$ fixes every coordinate but r. For $\mathrm{expr}(w) = e_1 \oplus e_2$ or $e_1 \odot e_2$, apply the hypothesis to e_1 and e_2 and use that substitution distributes over both operators while $\oplus$ and $\odot$ are computed pointwise from their operands' values. □

As a small remark, realize that the above lemma generalizes exactly as expected to a tuple of wires $\mathbf{w}$ by coordinatewise application.

Corollary 2.1 (Single-occurrence discharge)**.** *Suppose r occurs exactly once in the expression of a probe $\mathbf{w}$, as an operand of a sum $x = e \oplus r$ with $r \notin \mathrm{vars}(e)$, and let $\sigma = (r \mapsto e \oplus r)$. Then $\mathbf{w}\sigma$ is identically distributed to $\mathbf{w}$ over ρ uniform, and every occurrence of x in $\mathbf{w}\sigma$ has collapsed to the bare input r.*

Proof. Distributional equality is Lemma 2.1 applied coordinatewise to $\mathbf{w}$, using that $\widehat{\sigma}$ preserves the uniform distribution on $\{0,1\}^{\mathcal{R}}$. For the second claim, $x\sigma = e \oplus (e \oplus r) = r$ by $\mathbb{F}_2$-cancellation, and since r occurs nowhere else by hypothesis, no other part of $\mathbf{w}$ is touched. □

Remark 2.1. *Rewriting a probe by a substitution as above — whether in the general form of Lemma 2.1 or the single-occurrence case of Corollary 2.1 — leaves its distribution untouched. That is the key observation that derives the procedure to decide the security of a probe, by one substitution at a time, taking it to a form with no secret in sight. The single-occurrence case is the one the* `maskVerif` *tool (Barthe et al., 2019) calls* optimistic sampling. *A derivation is nothing more than such substitutions chained together; how that chain is searched for is taken up in the later chapters.*

2.5 Counting Complexity Classes

Before closing off this chapter, we mention two specific complexity classes that will come up later in the thesis. Let M be a nondeterministic polynomial-time Turing machine and write $\#M(x)$ for the number of accepting computation paths of M on input x; where NP asks only whether $\#M(x) > 0$, the classes we are concerned with look at the number itself.

- #P, introduced by Valiant (Valiant, 1979), is the class of *functions* of the form $x \mapsto \#M(x)$. It is a class of counting problems rather than decision problems. Its canonical member is #Sat, the number of satisfying assignments of a Boolean formula.
- $\mathrm{C}_{=}\mathrm{P}$, studied by Wagner (Wagner, 1986), is the class of *languages* $\{\, x \mid \#M(x) = g(x) \,\}$ for a polynomial-time computable g, specifying if the machine have *exactly* a prescribed number of accepting paths. This is a decision class, and it is the natural class of a yes-or-no question whose answer rests on a comparison of two counts. Its canonical complete problem is ExactSat, deciding whether a formula has exactly a given number of satisfying assignments.

Neither class is known to sit inside the polynomial hierarchy PH. Actually, both sit at least above it. Writing $\mathrm{P}^{\#\mathrm{P}}$ for the decision problems solvable in polynomial

time with an oracle for a counting function, two observations place these classes. A formula is unsatisfiable exactly when its count is 0, which is an instance of exact counting, so $\mathrm{coNP} \subseteq \mathrm{C_{=}P}$; and two oracle calls, a subtraction and a test against zero settle any exact-counting question, so $\mathrm{C_{=}P} \subseteq \mathrm{P}^{\#\mathrm{P}}$. Now, Toda's theorem states that $\mathrm{PH} \subseteq \mathrm{P}^{\#\mathrm{P}}$ (Toda, 1991), so a single counting oracle already decides every level of the hierarchy. In short,

$$\mathrm{P} \subseteq \mathrm{NP},\ \mathrm{coNP} \subseteq \mathrm{PH} \subseteq \mathrm{P}^{\#\mathrm{P}}, \qquad \mathrm{coNP} \subseteq \mathrm{C_{=}P} \subseteq \mathrm{P}^{\#\mathrm{P}},$$

hence a problem complete for either class is well beyond anything we should expect to solve exactly and efficiently.

Chapter 4 shows that the two problems underlying d-probing security (computing the law of one probe, and deciding whether two secrets leave it unchanged) fall into exactly these two classes.

3. RELATED WORK

Verifying that a masked circuit leaks nothing has been approached from several sides. The question is always the one of Chapter 2, asking whether the joint distribution of a set of wires vary with the secrets, but one may resolve it by rewriting an expression, by measuring a spectrum, by counting satisfying assignments, or by building the distributions explicitly. This chapter surveys those approaches, and places our work among them.

3.1 Language-Based Verification

The approach we build on views a masked circuit as a probabilistic program and treats a probe's leakage symbolically. Rather than estimating a distribution, it manipulates the expressions a set of wires compute until secrets have visibly cancelled. The line begins with (Barthe et al., 2015), and their framing of secret-independence as a property that can be established by distribution-preserving rewriting, is the largest influence of this work.

3.1.1 Verified Proofs of Higher-Order Masking

Barthe et al. (Barthe et al., 2015) gave a machine-checked proof of higher-order masking, built inside the EasyCrypt proof assistant and its probabilistic relational Hoare logic. Their checker decides whether a set of intermediate wires is independent of the secret by rewriting. The central method is called optimistic sampling, where

a random r occurring once, as $e \oplus r$, is replaced by a fresh uniform variable, since $e \oplus r$ is itself uniform when r does not appear in e. When substitutions alone fail to cancel all secrets, the expression is put into algebraic normal form so that products collapse, and independence can be declared if no secret variable occurs in that form. This is, in miniature, the rewrite engine of our Component I (Section 5.2). The optimistic sampling substitution is exactly the single-occurrence rule we inherit. However, where they go to ANF, we instead factor the expression (Section 6.1.3), which can reach products that normalisation miss.

3.1.2 Large Sets and the Exploration Algorithm

Deciding one set is not the only difficulty; indeed, there are $\sum_{i \leq d} \binom{|\mathcal{W}|}{i}$ of them. The language-based tools escape this enumeration with a large-set argument that the `maskVerif` tool (Barthe et al., 2019) made systematic. Secret-independence passes to subsets, so a proof about a large set of wires discharges every one of its subsets at once. So, the search tries to verify maximal sets and, on failure, splits and recurses. Our Component II (Section 5.3) is this exploration, and our contribution there is to make the split's exhaustiveness a neatly proved identity. The set-level Vandermonde partition of Section 5.3.3 certifies that no probe slips through such a recursion.

3.1.3 Witness Replay and Derivation Reuse

A certified set leaves behind a derivation, which is the sequence of rewrites that discharged it, and that derivation can be reused. `maskVerif` (Barthe et al., 2019) enlarges a verified set by replaying its derivation against neighbouring positions and keeping those that are also carried to a secret-free form after the same sequence of rewrites. So, one certification serves many wires. This is the direct ancestor of our Component III (Section 5.4.2). The reuse is imperfect for us, because our engine factors and factoring is not a trivially replayable step. Re-running a recorded derivation on a fresh wire misses the cancellations a factoring might have made possible. We therefore read the derivation as a measure-preserving coupling of the randomness (Chapter 4), and we test each candidate against it with the exact criterion of Section 5.4.3, which admits every wire replays also would.

3.1.4 MaskVerif and Physical Defaults

The value-based model, in which a probed wire leaks solely its logical value, is very optimistic about issues in hardware. A real gate *glitches*, briefly computing on unsettled inputs, and a register transition may leak two successive values. `maskVerif` (Barthe et al., 2019) captures these physical defaults, verifying masking in a glitch-extended probing model where a probe on a glitching gate carries information about all of that gate's stable inputs. We stay in the value-based model and do not treat glitches (Chapter 2), so those stronger models which do, bound the guarantee our checker delivers.

3.2 Fourier-Analytic Verification

A different approach does not rewrite at all. Observe that a set of wires is independent of the secret exactly when certain Fourier coefficients of the Boolean function they compute vanish, and one can verify masking by checking that spectral condition.

3.2.1 The Xiao–Massey Criterion and Approximations

The spectral implications of independence is widely known. Xiao and Massey (Xiao & Massey, 1988) characterised the correlation-immune Boolean functions as those whose Walsh spectrum vanishes at every nonzero point of Hamming weight at most t. So, a function is t-th order correlation-immune, that is, statistically independent of any t of its inputs, precisely under this condition. Bloem et al. (Bloem et al., 2018) carry this into masking verification. Their tool, `REBECCA`, propagates a conservative correlation set for each wire through the circuit. This set consists of monomials that might have a non-zero Fourier coefficient (indicating potential correlation), and the tool reports a leak when such a monomial mixes a secret with no fresh masking variable to blind it. Tracking which monomials can appear, rather than their exact coefficients, is the over-approximation that keeps the computation feasible, at the cost of the occasional false positives.

3.2.2 SAT-Based Realization and Its Cost

The correlation-immunity condition is discharged to a SAT solver. `REBECCA` (Bloem et al., 2018) reduces the search for a secret-dependent, unblinded correlation to a family of SAT queries whose number is polynomial in the count of gates and wires, and it runs in the glitch-extended model, making it one of the first tools to verify hardware masking under physical defaults. The correlation sets get increasingly convoluted with the masking order, so the propagation and the satisfiability search both climb as the order grows, and the over-approximation has to be loosened or tightened to trade precision against effort.

3.3 Other Approaches

Beyond rewriting and spectra, two further approaches are worth mentioning. One decides independence by educated counting, the other sidesteps the per-circuit question by reasoning about compositions of smaller gadgets.

3.3.1 Model Counting and SMT

Independence is a statement about a distribution, so one can check it by encoding the distribution and asking a solver. Eldib, Wang, and Schaumont (Eldib et al., 2014) verify software masking by expressing secret-independence as a satisfiability question and discharging it to an SMT solver. Gao et al. (Gao et al., 2019) makes the question quantitative, asking how far a wire is from independence through a model-counting notion of masking strength and scaling it with a refinement loop. `SILVER` (Knichel et al., 2020) goes fully exact, constructing the wires' joint distribution with reduced-ordered binary decision diagrams and testing statistical independence on it directly. These tools are exact where ours is incomplete, and the reason we do not follow them is the hardness of Chapter 4. Computing or comparing these distributions is #P- and $C_{=}P$-hard, so the exact route is taken with a heavy cost.

3.3.2 Compositional Reasoning and Relabeling

Probing security does not trivially compose. A gadget which is secure in isolation can leak once wired into another secure one. A body of work isolates stronger notions that allow compositions. Barthe et al. (Barthe et al., 2016) introduce strong non-interference, which handle output and internal probes separately and composes by construction, and Cassiers and Standaert (Cassiers & Standaert, 2020) refine this to a probe-isolating non-interference, under which gadgets compose nicely. Belaïd, Goudarzi, and Rivain (Belaïd et al., 2018) verify tight probing security directly, characterising when a masked circuit is secure with the least refreshing. These notions allow the assembly of large masked circuits from smaller components. We stay within plain probing security of a single circuit (Chapter 2) and do not pursue composition, so they are related to our work but do not overlap it.

3.4 Position of This Work

This thesis sits in the language-based lineage of Barthe et al. (Barthe et al., 2015, 2019). As they do, we decide a probe by rewriting it toward a secret-free form, we escape the subset enumeration by exploring large sets, and we reuse a certified derivation to enlarge a safe set. Three things set our treatment apart. First, we view a successful discharge as an explicit measure-preserving coupling of the randomness (Chapter 4) and admit neighbouring wires by an exact test through it. Second, we prove the exhaustiveness of the set-splitting, turning the traversal's soundness into the Vandermonde identity of Section 5.3.3 rather than an informal argument. Third, we add multiplicative factoring (Section 6.1.3) to reach randoms buried inside products. Unlike the Fourier and BDD tools we stay value-based and do not model glitches, and unlike the compositional notions we verify a whole circuit rather than assembling it from verified parts. Against the exact solvers and decision-diagram engines we decline to build the distribution at all, since Chapter 4 shows that to be intractable, and we certify constructively instead, which is sound but incomplete.

4. COUPLINGS AND PROBING SECURITY

4.1 The Complexity of Deciding Secret-Independence

Definition 2.7 phrases security as the condition that the law of a probe stay fixed as the secrets vary, and Section 2.5 already mentioned that this demand is actually a counting question. In this section we make this claim precise. Underneath a single probe sit two problems, computing its law and deciding whether varying the secret move it, and we determine each one's hardness. Computing the law is a #P-complete count, and the decision problem is a $\mathrm{C}_{=}\mathrm{P}$-complete comparison of such counts. This means that verifying even one probe by counting is complete for classes that sit above the entire polynomial hierarchy, which is the reason the thesis certifies security constructively without explicit counting.

Throughout, fix a circuit over $\mathbb{F}_2$ with secret inputs $\mathcal{S}$, random inputs $\mathcal{R}$, and public inputs $\mathcal{P}$, as usual, together with a probe $\mathbf{w} = (w_1, \ldots, w_k)$ of size $k \leq d$. Under a secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$ and a tape $\rho \in \{0,1\}^{\mathcal{R}}$, the probe takes the joint value $\mathbf{w}(\rho;\mathbf{a}) \in \{0,1\}^k$, with the publics held fixed.

4.1.1 From Independence to Distribution Counting

The law of a probe is assembled from integer counts. For an observation $o \in \{0,1\}^k$ write

$$N_{\mathbf{a}}(o) \;=\; \Big|\{\, \rho \in \{0,1\}^{\mathcal{R}} : \mathbf{w}(\rho;\mathbf{a}) = o \,\}\Big|,$$

the number of tapes on which the probe reads o under the secret $\mathbf{a}$. Since ρ is uniform, the law is $\mathcal{L}_{\mathbf{a}}(\mathbf{w})(o) = N_{\mathbf{a}}(o)/2^{|\mathcal{R}|}$, so the law is nothing but a histogram of

these counts. Two laws coincide exactly when their counts do,

$$\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}_{\mathbf{b}}(\mathbf{w}) \quad \Longleftrightarrow \quad N_{\mathbf{a}}(o) = N_{\mathbf{b}}(o) \ \text{ for every } o \in \{0,1\}^k.$$

Secret-independence (Definition 2.7) is this equality quantified over all pairs of secrets. So what looks like a statement about distributions is actually a statement about integers, that certain counts of satisfying tapes agree. This splits into two questions of different character. One asks for the count itself; the other asks only whether two counts are equal, a decision. We treat them in turn.

Definition 4.1 (The counting and distinguishing problems)**.** *Count takes a circuit, a probe* $\mathbf{w}$*, a secret* $\mathbf{a}$*, and an observation* o*, and returns the integer* $N_{\mathbf{a}}(o)$*. Distinguish takes a circuit, a probe* $\mathbf{w}$*, and two secret assignments* $\mathbf{a}, \mathbf{b}$*, and accepts exactly when* $\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}_{\mathbf{b}}(\mathbf{w})$*.*

4.1.2 Counting the Law Is #P-Complete

Recall that #P is Valiant's class of functions counting the accepting computations of a nondeterministic polynomial-time machine, equivalently it is the enumeration of the satisfying assignments of a polynomial-size Boolean predicate (Valiant, 1979).

Proposition 4.1 (Counting the law)**.** *Count is* #P*-complete.*

Proof. Membership. Fix $\mathbf{a}$, the publics, and o. The predicate $\rho \mapsto [\mathbf{w}(\rho; \mathbf{a}) = o]$ evaluates the circuit on the tape ρ and compares k output bits against o, all in time polynomial in the circuit size. Hence $N_{\mathbf{a}}(o)$ counts the tapes satisfying a polynomial-time predicate, so it is a #P function.

Hardness. We reduce from #SAT, the count of satisfying assignments of a Boolean formula, which is #P-complete (Valiant, 1979). Given $\varphi(x_1, \ldots, x_m)$, build the circuit that reads $x_1, \ldots, x_m$ as random inputs, has no secrets, and computes φ gate by gate, ending in a wire w with $\mathrm{expr}(w) = \varphi$; the connectives of φ are realised through $\oplus$ and $\odot$, which is possible because the two are universal over $\mathbb{F}_2$ (Section 2.1.3). The one-wire probe (w) then has $N(1) = |\{x : \varphi(x) = 1\}| = \#\varphi$, so any procedure for Count computes $\#\varphi$. The reduction is polynomial, hence Count is #P-hard. □

Notice that the hardness result uses no secret at all, so computing a probe's law is already intractable on the plainest circuits. That count is the object every exact

method must, explicity or implicitly, realize.

4.1.3 Deciding Independence is $\mathrm{C}_{=}\mathrm{P}$-Complete

DISTINGUISH never asks for a count outright. It asks only whether two counts are equal, and that equality has a class of its own. Following Wagner, $\mathrm{C}_{=}\mathrm{P}$ is the class of languages of the form "does a nondeterministic polynomial-time machine have exactly a given number of accepting paths" (Wagner, 1986). Another view comes from gap functions. Let GapP be the closure of #P under subtraction, so the differences $f - g$ of two counting functions (Fenner et al., 1994). Then $\mathrm{C}_{=}\mathrm{P}$ is exactly the class of vanishing sets of gap functions,

$$\Big\{ x \mid h(x) = 0 \Big\}, \qquad h \in \mathrm{GapP}.$$

Its canonical complete problem is EXACTSAT, deciding whether a formula has exactly a given number of satisfying assignments, equivalently one can also ask whether two formulas have the same number of satisfying assigmnets (Wagner, 1986).

Theorem 4.1 (Deciding independence)**.** *DISTINGUISH is* $\mathrm{C}_{=}\mathrm{P}$*-complete. Consequently deciding whether a probe is secret-independent is* $\mathrm{C}_{=}\mathrm{P}$*-hard.*

Proof. Membership. Put

$$f(C, \mathbf{w}, \mathbf{a}, \mathbf{b}) \;=\; \sum_{o \in \{0,1\}^k} \Big(N_{\mathbf{a}}(o) - N_{\mathbf{b}}(o) \Big)^2,$$

Where C is the circuit, each $N_{\mathbf{a}}$ is a #P function, and GapP is closed under subtraction and multiplication (Fenner et al., 1994). The sum ranges over $2^k \leq 2^d$ observations, which is a constant since the order d is fixed, thus f is a constant combination of gap functions and is itself in GapP. A sum of squares vanishes exactly when every term does, hence

$$\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}_{\mathbf{b}}(\mathbf{w}) \iff N_{\mathbf{a}}(o) = N_{\mathbf{b}}(o)\ \forall o \iff f(C, \mathbf{w}, \mathbf{a}, \mathbf{b}) = 0,$$

and DISTINGUISH is the vanishing set of the gap function f, so it lies in $\mathrm{C}_{=}\mathrm{P}$.

Hardness. We reduce from the equality form of EXACTSAT: given two formulas φ_0, φ_1 over $x_1, \ldots, x_m$, decide whether $\#\varphi_0 = \#\varphi_1$, which is $\mathrm{C}_{=}\mathrm{P}$-complete (Wagner, 1986). Build a circuit with a single secret bit s, the x_i as randoms, and a final wire

w with

$$\mathrm{expr}(w) \;=\; \varphi_0 \oplus \Big(s \odot (\varphi_0 \oplus \varphi_1)\Big),$$

again realising the formulas over $\oplus$ and $\odot$. Then w reads φ_0 when $s = 0$ and φ_1 when $s = 1$, so $N_{s=0}(1) = \#\varphi_0$ and $N_{s=1}(1) = \#\varphi_1$. The one-bit probe (w) satisfies $\mathcal{L}_0 = \mathcal{L}_1$ if and only if $\#\varphi_0 = \#\varphi_1$, so DISTINGUISH on this circuit with $\mathbf{a} = 0$, $\mathbf{b} = 1$ decides the equality. The construction is polynomial, so DISTINGUISH is $\mathrm{C}_{=}\mathrm{P}$-hard. One secret bit and one probed wire already suffice, so no feature of the masking is what makes the problem hard. □

4.1.4 What the Placements Mean for Verification

The two results are closely related. The law's entries are #P counts, and the security question is whether two of them coincide, which is the defining shape of the $\mathrm{C}_{=}\mathrm{P}$ class. The classes fall in the expected places. Every $\mathrm{C}_{=}\mathrm{P}$ query is settled by two calls to a counting oracle, followed by a subtraction, and a test against zero, so $\mathrm{C}_{=}\mathrm{P} \subseteq \mathrm{P}^{\#\mathrm{P}}$. Which means that the decision never costs more than the counting.

Another observation is the following: a formula is unsatisfiable exactly when its count is zero (as an instance of exact counting) so $\mathrm{coNP} \subseteq \mathrm{C}_{=}\mathrm{P}$. Hence DISTINGUISH is already coNP-hard, and no polynomial-time exact decision exists unless $\mathrm{P} = \mathrm{NP}$. Further showing us that deciding one probe exactly is very much intractable.

The full verification task only adds on this. d-probing security asks DISTINGUISH to answer "equal" for every one of the $\sum_{i \le d} \binom{|\mathcal{W}|}{i}$ probes and every pair of secrets. A universal quantifier over exponentially many instances of a $\mathrm{C}_{=}\mathrm{P}$-complete question.

So now we should be fairly certain that computing or equating these counts is out of reach, so we do not compute them. We certify security constructively instead, by exhibiting for a probe a measure-preserving relabelling of the randomness under which its secrets visibly cancel. Such a witness proves independence without ever counting, though at the price of completeness, since a probe can be independent for reasons no relabelling of our kind captures.

4.2 Couplings of Probabilistic Programs

The hardness of Section 4.1 rules out counting, so we need another way of certifying that two laws agree. The workaround idea comes from the fact that our two laws are not two unrelated distributions. They come from the same uniform random source.

Fix a probe $\mathbf{w}$ and two secret assignments $\mathbf{a}$ and $\mathbf{b}$. Both $\mathcal{L}_{\mathbf{a}}(\mathbf{w})$ and $\mathcal{L}_{\mathbf{b}}(\mathbf{w})$ are produced the same way, by drawing one uniform tape $\rho \in \{0,1\}^{\mathcal{R}}$ and reading the probe; the only difference between them is which secret is plugged in. So rather than computing the two distributions and comparing them, we can try to match up the tapes that produce them.

Suppose we can pair each tape ρ with a tape ρ' so that the run under $\mathbf{a}$ on ρ and the run under $\mathbf{b}$ on ρ' read exactly the same values,

$$\mathbf{w}(\rho;\mathbf{a}) \;=\; \mathbf{w}(\rho';\mathbf{b}),$$

and suppose the pairing $\rho \mapsto \rho'$ is a bijection of the tape space. Then the two laws are equal. Because, the bijection matches the tapes producing an observation o under $\mathbf{a}$ one-for-one with the tapes producing o under $\mathbf{b}$, so the counts $N_{\mathbf{a}}(o)$ and $N_{\mathbf{b}}(o)$ of Section 4.1.1 agree without either being computed explicitly. A relabelling of the randomness has replaced counting, which is exactly the trade we were looking for. Such a bijection is what we call a coupling.

The name is borrowed from probability theory, where a *coupling* of two distributions μ and ν on a set X is a joint distribution on $X \times X$ whose marginals are μ and ν, and where a coupling supported on the diagonal forces $\mu = \nu$. This general notion is what underlies the coupling-based program logics of Barthe et al. (Barthe et al., 2017).

4.2.1 Coupling Two Runs Under Different Secrets

Definition 4.2 (Coupling function)**.** *Let* $\mathbf{w}$ *be a probe and* $\mathbf{a}, \mathbf{b} \in \{0,1\}^{\mathcal{S}}$ *two secret assignments. A* coupling function *from* $\mathbf{a}$ *to* $\mathbf{b}$ *for* $\mathbf{w}$ *is a bijection* $\varphi : \{0,1\}^{\mathcal{R}} \to \{0,1\}^{\mathcal{R}}$ *with*

$$\mathbf{w}(\rho;\mathbf{a}) \;=\; \mathbf{w}\big(\varphi(\rho);\mathbf{b}\big) \qquad \textit{for every } \rho \in \{0,1\}^{\mathcal{R}}.$$

A bijection of the finite set $\{0,1\}^{\mathcal{R}}$ automatically preserves its uniform law, so "measure-preserving" actually just means bijectivity here, as Section 2.1.2 already noted. What the identity demands is the key point. It says φ relabels the tapes so that a run under $\mathbf{a}$ is reproduced, observation for observation, by a run under $\mathbf{b}$. Such a φ is precisely a diagonal coupling of the two laws, realised by the map $\rho \mapsto \Big(\mathbf{w}(\rho;\mathbf{a}), \mathbf{w}(\varphi(\rho);\mathbf{b})\Big)$, whose image lies on the diagonal.

Collecting one function per ordered pair of secrets gives a family $\Phi = \{\,\Phi_{\mathbf{a},\mathbf{b}}\,\}_{\mathbf{a},\mathbf{b}}$ of coupling functions. We may always take $\Phi_{\mathbf{a},\mathbf{a}} = \mathrm{id}$, $\Phi_{\mathbf{b},\mathbf{a}} = \Phi_{\mathbf{a},\mathbf{b}}^{-1}$, and $\Phi_{\mathbf{b},\mathbf{c}} \circ \Phi_{\mathbf{a},\mathbf{b}} = \Phi_{\mathbf{a},\mathbf{c}}$, so the family is a groupoid on the secret assignments and one row $\{\Phi_{\mathbf{a}_0,.}\}$ generates the rest.

4.3 Probing Security via Couplings

The definition is engineered so that its existence and secret-independence are one and the same statement.

Theorem 4.2 (Security is a coupling)**.** *A probe* $\mathbf{w}$ *is secret-independent if and only if for every ordered pair of secret assignments* $\mathbf{a},\mathbf{b}$ *there is a coupling function from* $\mathbf{a}$ *to* $\mathbf{b}$ *for* $\mathbf{w}$*.*

Proof. Suppose φ is a coupling function from $\mathbf{a}$ to $\mathbf{b}$. For any observation o, the identity $\mathbf{w}(\rho;\mathbf{a}) = \mathbf{w}(\varphi(\rho);\mathbf{b})$ and the bijectivity of φ give

$$N_{\mathbf{a}}(o) = \Big|\{\rho : \mathbf{w}(\rho;\mathbf{a}) = o\}\Big| = \Big|\{\rho : \mathbf{w}(\varphi(\rho);\mathbf{b}) = o\}\Big| = \Big|\{\rho' : \mathbf{w}(\rho';\mathbf{b}) = o\}\Big| = N_{\mathbf{b}}(o),$$

reindexing by $\rho' = \varphi(\rho)$. Hence $\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}_{\mathbf{b}}(\mathbf{w})$, and if this holds for every pair then $\mathbf{w}$ is secret-independent by definition.

For the other way, suppose $\mathcal{L}_{\mathbf{a}}(\mathbf{w}) = \mathcal{L}_{\mathbf{b}}(\mathbf{w})$, i.e. $N_{\mathbf{a}}(o) = N_{\mathbf{b}}(o)$ for every o. The sets $F_{\mathbf{a}}(o) = \{\rho \mid \mathbf{w}(\rho;\mathbf{a}) = o\}$ partition $\{0,1\}^{\mathcal{R}}$ as o ranges over the observations, and likewise the $F_{\mathbf{b}}(o)$. Corresponding blocks have equal size, so we may pick a bijection $F_{\mathbf{a}}(o) \to F_{\mathbf{b}}(o)$ for each o and take their union φ. It is a bijection of $\{0,1\}^{\mathcal{R}}$, and by construction it carries each $\rho \in F_{\mathbf{a}}(o)$ to some $\rho' \in F_{\mathbf{b}}(o)$, so $\mathbf{w}(\varphi(\rho);\mathbf{b}) = o = \mathbf{w}(\rho;\mathbf{a})$. So, φ is a coupling function from $\mathbf{a}$ to $\mathbf{b}$. □

4.3.1 d-Probing Security as a Family of Couplings

Applying Theorem 4.2 to every probe lifts the equivalence to the whole circuit. It is d-probing secure exactly when every probe of size at most d admits a family of coupling functions, one per ordered pair of secrets. Two runs with differing secrets give any d wires the same law precisely because a coupling can relabel the randomness of one into the randomness of the other, wire for wire.

The construction of such a coupling, assuming that the probe does not leak, turns the assumption into a coupling by matching fibres block by block, and to do so it needs the counts $N_{\mathbf{a}}(o)$, which are the very quantities Section 4.1 proved to be intractable. So the equivalence, is an existence statement and not really a recipe. It guarantees a coupling without telling us how to find one nor how to check it efficiently. Supplying such an explicit witness is the task of the construction we explain below.

4.4 Substitutions as a Coupling Construction

The randomness substitutions of Section 2.4 can become the building blocks of an explicit coupling. Recall that a substitution $\sigma = (r \mapsto e \oplus r)$ induces a bijection $\widehat{\sigma}$ of $\{0,1\}^{\mathcal{R}}$, with $(w\sigma)(\rho) = w(\widehat{\sigma}(\rho))$ for every wire (Lemma 2.1). Each such $\widehat{\sigma}$ is a single relabelling of the tape, and relabellings of this shape compose. A finite chain of substitutions therefore composes into a bijection of the randomness, and it is this composite that we will exhibit as the witness of Theorem 4.2.

4.4.1 The Derivation of an Explicit Coupling Witness

Definition 4.3 (Derivation)**.** *A* derivation *for a probe* $\mathbf{w}$ *is a finite sequence of substitutions*

$$\sigma_1 = (r_1 \mapsto e_1 \oplus r_1),\ \ldots,\ \sigma_K = (r_K \mapsto e_K \oplus r_K), \qquad r_i \notin \mathrm{vars}(e_i),$$

applied in order. It ends secret-free *when the resulting expression*

$$\overline{\mathbf{w}} \;=\; \mathbf{w}\sigma_1\sigma_2\cdots\sigma_K$$

is secret-free in the sense of Notation 2.1, i.e. no secret variable occurs in it.

Before we construct our coupling from the derivation we mention two small details. First, Lemma 2.1 was stated with the secrets held fixed, however now we let them vary, and we must record how the pieces depend on them. Second, the context e_i of a substitution may itself mention secret variables, so the shift it applies to the r_i-coordinate is not one map but one map per secret assignment. We write $\widehat{\sigma_i}^{\mathbf{a}}$ for the bijection induced by σ_i under the secret assignment $\mathbf{a}$, the one that shifts the r_i-coordinate by $e_i(\rho;\mathbf{a})$.

Composing these under one secret assignment $\mathbf{a}$, let

$$\varphi_{\mathbf{a}} \;=\; \widehat{\sigma_1}^{\mathbf{a}} \circ \widehat{\sigma_2}^{\mathbf{a}} \circ \cdots \circ \widehat{\sigma_K}^{\mathbf{a}},$$

a bijection of $\{0,1\}^{\mathcal{R}}$, being a composition of bijections. The derivation is thus a purely syntactic list of rewrite steps, which yields an actual tape bijection only once a secret assignment is supplied. The theorem below says that these bijections fit together into a coupling.

Theorem 4.3 (Derivations yield couplings)**.** *Let $\sigma_1,\ldots,\sigma_K$ be a derivation for a probe $\mathbf{w}$ that ends secret-free, and let $\varphi_{\mathbf{a}}$ be as above. Then for every ordered pair of secret assignments*

$$\Phi_{\mathbf{a},\mathbf{b}} \;=\; \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$$

is a coupling function from $\mathbf{a}$ to $\mathbf{b}$ for $\mathbf{w}$ in the sense of Definition 4.2. Consequently $\mathbf{w}$ is secret-independent.

Proof. Fix a secret assignment $\mathbf{a}$ for the moment. Lemma 2.1 says that rewriting the expression by one substitution is the same as moving the tape by the bijection it induces. Doing this once per step and composing gives

$$\mathbf{w}\Big(\varphi_{\mathbf{a}}(\rho);\mathbf{a}\Big) \;=\; \overline{\mathbf{w}}(\rho;\mathbf{a}) \qquad \text{for every tape } \rho.$$

This can be shown formally with an induction on K.

Since $\overline{\mathbf{w}}$ is secret-free, evaluating it never reads the secret assignment, so we may

write $\overline{\mathbf{w}}(\rho)$ and read the identity above at $\mathbf{a}$ and at $\mathbf{b}$, as follows:

$$\mathbf{w}\Big(\varphi_{\mathbf{a}}(\rho);\mathbf{a}\Big) \;=\; \overline{\mathbf{w}}(\rho) \;=\; \mathbf{w}\Big(\varphi_{\mathbf{b}}(\rho);\mathbf{b}\Big).$$

Both runs are relabelled views of the same secret-free expression, and it only remains to link them to each other. As $\varphi_{\mathbf{a}}$ is a bijection we may put $\varphi_{\mathbf{a}}^{-1}(\rho)$ in place of ρ, which leaves $\mathbf{w}(\rho;\mathbf{a})$ on the left and gives

$$\mathbf{w}(\rho;\mathbf{a}) \;=\; \mathbf{w}\Big(\Phi_{\mathbf{a},\mathbf{b}}(\rho);\mathbf{b}\Big) \qquad \text{for every } \rho,$$

with $\Phi_{\mathbf{a},\mathbf{b}} = \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$ being a bijection, as a composition of two. This is exactly Definition 4.2. The pair $\mathbf{a},\mathbf{b}$ was arbitrary, so Theorem 4.2 applies and $\mathbf{w}$ is secret-independent. □

The family $\{\Phi_{\mathbf{a},\mathbf{b}}\}$ built this way satisfies the groupoid identities of Section 4.2.1 for free, since $\Phi_{\mathbf{a},\mathbf{a}} = \mathrm{id}$, $\Phi_{\mathbf{b},\mathbf{a}} = \Phi_{\mathbf{a},\mathbf{b}}^{-1}$ and $\Phi_{\mathbf{b},\mathbf{c}} \circ \Phi_{\mathbf{a},\mathbf{b}} = \Phi_{\mathbf{a},\mathbf{c}}$ all follow from the shape of $\varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$.

This is the constructive certificate that we wished for. It is explicit, constructed straight off the recorded substitutions, and it is checkable without ever counting a fibre. How such a derivation is searched for, recorded, and then reused to certify many wires at once is the subject of the next chapter (Chapter 5).

5. DESIGN OF COPPULA

5.1 Pipeline Overview

Coppula takes an arithmetic circuit over $\mathbb{F}_2$, where the wires are partitioned into secret, random, and public inputs (Chapter 2), together with a probing order d, and returns a verdict: the circuit is d-probing secure, or it is not and a witnessing probe is handed back. Underneath sits the single question of Section 2.3.2, is every probe (Definition 2.6) secret-independent, and two difficulties have to be resolved for an answer. Deciding one single probe is already non-trivial, it is a statement about a distribution over the randomness space and there are combinatorially many probes to decide. The design addresses both difficulties.

Three components divide the labour. *Component I*, the rewrite engine (Section 5.2), is the oracle for a single probe. It decides secret-independence not by summing over a distribution but by rewriting the probe's expression, one randomness substitution at a time, into a form with no secret in sight (Section 2.4). *Component II*, the subset-space traversal (Section 5.3), is the driver. It sweeps every d-subset of the wires without enumerating them one by one, in the recursive set-splitting style of `maskVerif` (Barthe et al., 2019). *Component III*, the probe-set extension (Section 5.4), is the amplifier. It turns the certificate for one probe into a certificate for a strictly larger safe set of wires, so that a single discharge settles many probes at once.

All three components use couplings. When Component II reaches a batch of probes it cannot dismiss as a whole, it asks Component I to certify a representative; a successful discharge returns not just a secure verdict but a bijective relabelling of the randomness space (a coupling function) that witnesses it. Component III substitutes that coupling in the circuit's other wires and gathers those that are blinded (rid of

secrets) into the safe set. Component II then splits the batch about that safe set and recurses, with the safe part already certified. Soundness of the whole rests on Component I alone (Corollary 5.1); Components II and III only utilize its verdicts, adding neither unsoundness nor any incompleteness of their own.

The rest of the chapter takes the three components in turn, asking what each computes and why it is correct. The concrete data structures that make them time- and memory-wise efficient are explained in Chapter 6.

5.2 Component I: The Rewrite Engine

Given a probe $\mathbf{w} = (w_1, \ldots, w_k)$, $k \leq d$ — a tuple of wires of the circuit — the rewrite engine decides whether $\mathbf{w}$ is secret-independent. Throughout this chapter a wire is handled as the $\mathbb{F}_2$ polynomial that defines it, its expression $\mathrm{expr}(w_i)$ is read off from the circuit graph (Definition 2.4); the engine only ever inspects and rewrites that syntax, while the wire functions of Definition 2.5 are what the soundness proofs appeal to. The engine works constructively, rather than reasoning about probability distributions directly. It searches for a finite sequence of applications of Lemma 2.1 (Section 2.4) after which $\mathbf{w}$'s form is secret-free in the sense of Notation 2.1, so no secret variable occurs in the expression of any w_i. Termination with a secret-free image certifies $\mathbf{w}$. Corollary 5.1 in Section 5.2.6 makes precise why this certificate is sound.

The engine has three tiers of rewriting rules, increasing in generality and cost, summarized in Algorithm 1 and mirrored in the structure of this section. Its single entry point is the routine CheckProbe of Algorithm 1, which tries the tiers in turn and returns a verdict together with the derivation T that Component III later uses. SimpleRewrite (Section 5.2.2) handles the common case of every random occurring at most once, cheaply. It is tried first on the unfactored probe and, on failure, once more after the multiplicative factoring of Section 6.1.3 has had a chance to expose additional single-occurrence randoms. This factoring & rewriting is the second tier. GeneralRewrite (Section 5.2.3) is the complete fallback for the remaining case, for random variables occurring more than once. It substitutes a random by its context wherever the random occurs, interleaving factoring lazily whenever both of its own find-rules stall. Section 5.2.6 closes the section by viewing the whole derivation as an explicit, replayable bijection on the randomness space. Which becomes the Φ

that Component III (Section 5.4) later utilizes in every other wire in the circuit.

5.2.1 The Record T

A piece of data the algorithm keeps track of through every routine is T, so we clarify its contents. It is a list of pairs, initially empty and only ever appended to,

$$T \;=\; \Big\langle (r_1,t_1),\ldots,(r_K,t_K) \Big\rangle, \qquad t_i \;=\; e_i \oplus r_i,$$

where the entry (r_i, t_i) records that the engine replaced the random r_i by the expression t_i, i.e. that it applied the substitution $\sigma_i = (r_i \mapsto t_i)$ of Definition 2.8. The pairs appear in the order the steps were taken. Both rewriting routines push in the same format, SimpleRewrite pushes (r, x) for the unique additive parent $x = e \oplus r$ it collapses, GeneralRewrite pushes (r, t) for the target $t = e \oplus r$ it substitutes. So T is exactly a derivation in the sense of Definition 4.3.

5.2.2 The Simple Rewrite Rule

The cheapest tier discharges the pattern Corollary 2.1 addresses directly, where a random occurs exactly once below the probe, additively (as an operand of an XOR gate). Such a random is a simple-rule candidate, provided it is not itself a probe root. Writing $x = e \oplus r$ for its unique parent, the corollary lets us relabel x as a fresh random and drop r without ever looking at the context e. The pattern certifies that x, jointly with everything independent of r, is distributed as a fresh, independent randomness. The tier applies this rewrite repeatedly, and with each rewrite it possibly creates further randoms fitting the pattern, until either no secret variable is reachable from the probe (success) or no candidate remains (fallback).

The exclusion of probe roots is a soundness condition. A root's value is what the adversary observes, so it is fixed, and relabelling it would claim independence for something the probe actually exposes.

The order in which rewrites fire is based on the multiplicative depth of the random variable. Randoms closest to a root get rewritten first.

Algorithm 1 Discharging a probe (Component I)

function CHECKPROBE($\mathbf{w}$)
Require: probe $\mathbf{w}$ (Definition 2.6), handled as the tuple of expressions $\mathrm{expr}(w_i)$
Ensure: a verdict, Secure or Insecure, with the recorded derivation T (the shears of §5.2.6)
    $(ok, T) \leftarrow$ SIMPLEREWRITE($\mathbf{w}, \langle\rangle$) ▷ Tier 1: unfactored
    if ok **then return** (Secure, T)
    end if
    $\mathbf{w}_f \leftarrow$ FACTOR($\mathbf{w}$) ▷ Tier 2: factor (§6.1.3)
    $(ok, T) \leftarrow$ SIMPLEREWRITE($\mathbf{w}_f, \langle\rangle$) ▷ retry the simple rule
    if ok **then return** (Secure, T)
    end if
    $(ok, T) \leftarrow$ GENERALREWRITE($\mathbf{w}, \emptyset, \mathit{fuel}, \mathit{false}, \langle\rangle$) ▷ Tier 3: complete fallback
    return ok ? (Secure, T) : (Insecure, T)
end function

function SIMPLEREWRITE($\mathbf{w}, T$)
    $\mathit{todo} \leftarrow \{\, r \in \mathrm{vars}(\mathbf{w}) : r$ occurs once, additively, $r \notin \mathbf{w} \,\}$
    $\mathit{todo} \leftarrow$ SORTBYMULDEPTH(todo) ▷ shallowest multiplicative depth first
    while $\mathit{todo} \neq \langle\rangle$ **and not** SECRETFREE($\mathbf{w}$)[a] **do**
        $r \leftarrow$ pop shallowest element of todo
        $x \leftarrow$ unique additive parent of r ▷ $x = e \oplus r$
        mark x as a fresh random leaf
        $T \leftarrow T.\mathrm{push}(r, x)$
        drop r ▷ may expose a new single-occurrence random for todo
    end while
    return (SECRETFREE($\mathbf{w}$), T)
end function

function GENERALREWRITE($\mathbf{w}, \mathit{used}, \mathit{fuel}, \mathit{factored}, T$)
    if SECRETFREE($\mathbf{w}$) **then return** (true, T)
    end if
    if $\mathit{fuel} = 0$ **then return** (false, T)
    end if
    $r \leftarrow$ FINDSIMPLE($\mathbf{w}$) ▷ single-occurrence, additive, non-root
    if $r = \bot$ **then** $r \leftarrow$ FINDGENERAL($\mathbf{w}, \mathit{used}$) ▷ multi-occurrence, at least one additive instance
    end if
    if $r = \bot$ **then**
        if $\mathit{factored}$ **then return** (false, T)
        end if
        $\mathbf{w}_f \leftarrow$ FACTOR($\mathbf{w}$) ▷ lazy factoring (§6.1.3)
        return GENERALREWRITE($\mathbf{w}_f, \mathit{used}, \mathit{fuel}, \mathit{true}, T$)
    end if
    $x \leftarrow$ chosen additive parent of r; $e \leftarrow \bigoplus(\mathrm{children}(x) \setminus \{r\})$ ▷ $r \notin \mathrm{vars}(e)$
    $t \leftarrow e \oplus r$; $\mathbf{w}' \leftarrow \mathbf{w}[r \mapsto t]$ ▷ Lemma 2.1
    return GENERALREWRITE($\mathbf{w}', \mathit{used} \cup \{r\}, \mathit{fuel} - 1, \mathit{false}, T.\mathrm{push}(r, t)$)
end function

a. SECRETFREE($\mathbf{w}$) returns true exactly when no secret variable occurs in the current expression of any wire of $\mathbf{w}$.

This tier is sound, since each step is a direct instance of Corollary 2.1, but it is incomplete. It has nothing to offer for a random occurring twice, such as a mask reused across two cross-terms of a masked multiplication. Section 5.2.3's fallback handles such cases.

5.2.3 Fallback: General Substitution

GeneralRewrite handles randoms that occur more than once by performing the substitution of Lemma 2.1 literally, rather than only its single-occurrence case (Corollary 2.1). It picks a random r and an additive occurrence $x_1 = e \oplus r$ with $r \notin \text{vars}(e)$, then rewrites every root by substituting $r \mapsto e \oplus r$ throughout. Cancellation in $\mathbb{F}_2$ collapses the chosen occurrence x_1 to the bare leaf r, exactly as in Corollary 2.1, while every other occurrence of r absorbs e instead of vanishing, cancelling further if it happens to share structure with e. The loop succeeds once no secret variable remains in the rewritten roots.

Each round re-examines the current probe for an applicable substitution. FindSimple looks first for the same single-occurrence, non-root pattern as Section 5.2.2, since a general substitution can turn a multi-occurrence random into a single-occurrence one; it also prefers the shallowest candidate. Failing that, FindGeneral looks for any random, not yet used by a previous general step, occurring at least twice with some additive parent x_1 whose other children do not themselves depend on it, with the side condition $r \notin \text{vars}(e)$ that Lemma 2.1 requires.

When both find-rules stall on the probe's current form, the engine factors (Section 6.1.3) once and retries, rather than declaring failure immediately. Factoring can move a random from a purely multiplicative position, where neither find-rule can see it, into an additive one. Factoring is symmetrically not tried before the simple rule either. Algorithm 1 always tries SimpleRewrite unfactored first, because factoring can destroy a single-occurrence opportunity the unfactored form exposes, so trying the cheap, unfactored tier first is not just faster on the common case but strictly more complete.

FindGeneral is needed precisely when a random occurs additively in two places that do not telescope into one another under the simple rule. Suppose r occurs as an operand of $x_1 = e_1 \oplus r$ in one root and as $x_2 = e_2 \oplus r$ in another, with $r \notin \text{vars}(e_1) \cup \text{vars}(e_2)$. Substituting through x_1, i.e. $r \mapsto e_1 \oplus r$, turns x_1 into $e_1 \oplus (e_1 \oplus r) = r$ by

cancellation, as in Corollary 2.1, while the same substitution turns x_2 into $e_2 \oplus e_1 \oplus r$. If $e_1 \oplus e_2$ is itself secret-free, both roots are secret-free after this one step; otherwise the loop continues, substituting further or factoring to expose a new opportunity. This is the shape of the case a masked multiplication typically produces, where a single random masks two different cross-terms: neither term's random is disposable on its own, but the substitution above can still cancel the secret-dependent part they share. Whether a given instance is discharged this way, and in how many rounds, depends entirely on how much of $e_1 \oplus e_2$ the rest of the derivation can still cancel; the loop offers no guarantee beyond trying. Even granting unlimited fuel, it only composes single-coordinate shears $\widehat{\sigma}$, so it can only certify secret-independence witnessed by a chain of substitutions, possibly interleaved with factoring.

Termination is only argued informally, not proved. Each general substitution retires its random into *used* and it is never repeated, bounding the number of general steps by $|\mathcal{R}|$; between general steps, simple substitutions strictly reduce the candidate pool; factoring is retried at most once per stall before the engine gives up on the current form. Interleaving factoring with two mutually triggering find-rules resists a clean potential-function argument, so a generous fuel counter backstops the loop instead. Exhausting fuel is treated as failure, which is sound (Corollary 5.1 is never invoked on a fuel-out) but not necessarily tight. So, a fuel-out reports the probe as though it were genuinely insecure, without exhibiting an actual distinguishing observation.

5.2.4 The Factoring Step

FACTOR appears twice in Algorithm 1, as the second tier and again lazily inside the third, so we explain what it actually does.

Both find-rules look for a random in *additive* position, as an operand of a sum, because that is the only shape Lemma 2.1 rewrites. A random sitting only inside products is therefore invisible to them, even when the probe could in fact be discharged. FACTOR reshapes the expression to expose such a random, by applying the distributive law right to left over a sum of products,

$$(f \odot a) \oplus (f \odot b) \oplus \mathit{rest} = \Big(f \odot (a \oplus b)\Big) \oplus \mathit{rest},$$

pulling out a factor f shared by two or more summands. The inner sum $a \oplus b$ is new

additive structure, and a random trapped in it becomes reachable. Which factor to pull out is a heuristic choice, and the pass is run repeatedly and bottom-up; Section 6.1.3 gives further implementation details.

What matters here is that FACTOR is not a rewrite in the sense of Section 2.4. It substitutes nothing, retires no random, and pushes nothing onto T. The displayed equation is an identity over $\mathbb{F}_2$, so the expression changes while the function it denotes does not. Factoring only changes what the find-rules can see.

5.2.5 Soundness of the Rules

We can now prove the soundess of Algorithm 1 one routine at a time. Throughout, "T is a derivation for $\mathbf{w}$" means in the sense of Definition 4.3, as we have discussed in Section 5.2.1.

Lemma 5.1 (FACTOR preserves values)**.** *For every probe* $\mathbf{w}$ *and every secret assignment* $\mathbf{a}$*,* FACTOR$(\mathbf{w})$ *and* $\mathbf{w}$ *denote the same function:* FACTOR$(\mathbf{w})(\rho;\mathbf{a}) = \mathbf{w}(\rho;\mathbf{a})$ *for every tape* ρ*.*

This is clear.

Lemma 5.2 (SIMPLEREWRITE is sound)**.** *If* SIMPLEREWRITE$(\mathbf{w}, \langle\rangle)$ *returns* $(true, T)$*, then* T *is a derivation for* $\mathbf{w}$ *that ends in a secret-free form.*

Proof. The loop pushes an entry only for a random r drawn from *todo*, which by construction occurs exactly once in $\mathbf{w}$, additively, and is not a probe root. Its unique additive parent $x = e \oplus r$ satisfies $r \notin \mathrm{vars}(e)$, so $\sigma = (r \mapsto e \oplus r)$ is a substitution in the sense of Definition 2.8 and the step is an instance of Corollary 2.1. Relabelling x as a fresh random and dropping r is simply the effect of applying σ. Each iteration therefore appends one legitimate substitution to the record. The loop returns *true* only when SECRETFREE$(\mathbf{w})$ holds, that is when the current expression is secret-free. □

Lemma 5.3 (GENERALREWRITE is sound)**.** *If a call to* GENERALREWRITE *on* $\mathbf{w}$ *with record* T_0 *returns* $(true, T)$*, then* T *extends* T_0 *to a derivation for* $\mathbf{w}$ *that ends in a secret-free form.*

Proof. We do an induction on the recursion. The base case returns *true* with $T = T_0$ only when $\mathbf{w}$ is already secret-free. A substitution step selects r and an additive

parent $x = e \oplus r$ with $r \notin \mathrm{vars}(e)$ (conditions both FindSimple and FindGeneral enforce) so $\sigma = (r \mapsto e \oplus r)$ is again a substitution in the sense of Definition 2.8, and $\mathbf{w}[r \mapsto t]$ is its application (Lemma 2.1). The induction hypothesis applies to the recursive call with the record extended by (r, t). A factoring step recurses on Factor($\mathbf{w}$) with T untouched, which is sound because Lemma 5.1 leaves the denoted function unchanged, so a derivation for the factored form is a derivation for $\mathbf{w}$. □

5.2.6 Relation to the Coupling-Construction View

A discharge does more than answer Secure/Insecure. Read the right way, it hands back the object Chapter 4 studies: a coupling for the probe, in the sense of Definition 4.2. All the work has been done there, and this subsection only has to observe that what the engine records is a derivation.

Discharging a probe amounts to finding a derivation (Definition 4.3), a finite sequence of substitutions $\sigma_1, \ldots, \sigma_K$ after which no secret variable remains. It does not matter which of the three tiers proposed a given step, nor whether the random it retires occurred once or many times: each σ_i is a substitution in the sense of Definition 2.8, hence a shear $\widehat{\sigma_i}$ (Lemma 2.1), and the coupling is assembled from these shears and from nothing else. That is why the engine can record every step in the same way, and why the construction of Section 4.4.1 never needs to know how an individual rewrite was found.

Corollary 5.1 (Soundness of discharge)**.** *If* CheckProbe($\mathbf{w}$) *(Algorithm 1) returns* Secure*, then* $\mathbf{w}$ *is secret-independent, and the recorded derivation* T *realises a coupling* $\Phi_{\mathbf{a},\mathbf{b}} = \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$ *of* $\mathbf{w}$*.*

Proof. CheckProbe returns Secure in three places, and each is covered. The first and third are a *true* from SimpleRewrite and from GeneralRewrite on $\mathbf{w}$ itself, so Lemma 5.2 and Lemma 5.3 make T a derivation for $\mathbf{w}$ ending secret-free. The second is a *true* from SimpleRewrite on the factored probe $\mathbf{w}_f$, which gives a derivation for $\mathbf{w}_f$; by Lemma 5.1 $\mathbf{w}_f$ denotes the same function as $\mathbf{w}$, so it is a derivation for $\mathbf{w}$ as well. In every case T is a derivation for $\mathbf{w}$ that ends secret-free, and Theorem 4.3 applies. □

5.3 Component II: Traversing the Subset Space

d-probing security asks that *every* set of at most d wires be secret-independent (Section 2.3.2). Read literally, that is $\sum_{i=1}^{d} \binom{N}{i}$ separate questions for a circuit of N wires. Component II is the bookkeeping that keeps the question exact while never enumerating a subset redundantly.

Two elementary observations are helpful to understand the traversal. First, we should see that secret-independence passes to subsets, specifically a subset of a secret-independent set is a marginal of a secret-independent law, hence itself is secret-independent. Certifying a set of wires therefore certifies all of its subsets in one stroke. Second, it is enough to look at probes of size *exactly* d: a smaller probe is a marginal of some size-d probe (whenever $N \geq d$, which always holds here), so it is covered once the size-d probes are. The task is thus to certify every d-element subset of the wire set, in batches as large as possible rather than one at a time.

5.3.1 The Representation of the Subset Space

The traversal never holds an explicit list of subsets; it holds a compact description of a whole family of them at once. A *factor* is a pair (c, W) of a count $c \in \mathbb{N}$ and a wire set W, standing for the family $\binom{W}{c}$ of all c-element subsets of W. A *worklist* is a finite sequence of factors

$$wl = \Big\langle (c_1, W_1), \ldots, (c_m, W_m) \Big\rangle, \qquad W_1, \ldots, W_m \text{ pairwise disjoint},$$

denoting the probes obtained by drawing c_j wires from each W_j and taking the union:

$$[\![wl]\!] \;=\; \Big\{ S_1 \cup \cdots \cup S_m \;:\; S_j \in \tbinom{W_j}{c_j} \Big\}, \qquad \big|[\![wl]\!]\big| \;=\; \prod_{j=1}^{m} \binom{|W_j|}{c_j}.$$

Disjointness of the W_j makes that union a disjoint one, so every probe has size $\sum_j c_j$ and none is described twice. Two vacaous worklists items are worth mentioning: if some factor asks for more than it holds, $|W_j| < c_j$, then $\binom{W_j}{c_j} = \emptyset$ and $[\![wl]\!]$ is empty; if every $c_j = 0$, the sole member is the empty probe. Either way there is nothing to certify. The verification task itself is the single-factor worklist $\big\langle (d, \mathcal{W}) \big\rangle$, whose

denotation is $\binom{\mathcal{W}}{d}$, i.e. all d-subsets of the circuit's wires $\mathcal{W}$.

Algorithm 2 sweeps this representation with three moves: discharge the union of the whole worklist and, on success, prune the branch; otherwise certify a representative probe and extend it to a safe set; then split each factor about that safe set and recurse on the pieces. The next two subsections treat the moves prune-and-extend and split, and presents a proof that the split loses nothing. In this section the extension part is kept ambigious, as its inner workings will be descredin in Section 5.4.

Algorithm 2 Traversing the subset space (Component II)

function CheckAll($wl = \langle (c_1, W_1), \ldots, (c_m, W_m) \rangle$)
  if $[\![wl]\!] = \emptyset$ **then return** Secure ▷ some $|W_j| < c_j$, or every $c_j = 0$
  end if
  $U \leftarrow W_1 \cup \cdots \cup W_m$
  $(v, T) \leftarrow$ CheckProbe(U) ▷ large-set pruning (§5.3.2)
  if $v =$ Secure **then return** Secure
  end if
  $\mathbf{w} \leftarrow$ a representative of $[\![wl]\!]$ ▷ c_j wires from each W_j
  $(v, T) \leftarrow$ CheckProbe($\mathbf{w}$) ▷ Component I (§5.2)
  if $v =$ Insecure **then return** Insecure
  end if
  safe $\leftarrow$ Extend($\mathbf{w}, T, U$) ▷ Component III (§5.4); $\mathbf{w} \subseteq$ safe $\subseteq U$
  for all $(i_1, \ldots, i_m) \in \prod_j \{0, \ldots, c_j\}$ **with** $(i_j)_j \neq (c_j)_j$ **do**
    $wl' \leftarrow \left\langle (i_j,\ W_j \cap \text{safe}),\ \ (c_j - i_j,\ W_j \setminus \text{safe}) \right\rangle_{j=1}^{m}$ ▷ drop zero-count parts
    $r \leftarrow$ CheckAll(wl')
    if $r =$ Insecure **then return** Insecure
    end if
  end for
  return Secure
end function

5.3.2 Large-Set Pruning

Before splitting anything, the traversal tries to discharge the whole batch at once. Let $U = W_1 \cup \cdots \cup W_m$ be every wire the worklist can reach, and hand U — as a single large probe — to the rewrite engine of Component I. If U is certified secret-independent, so is every subset of U, and in particular every probe in $[\![wl]\!]$, meaning that the branch is secure and the traversal returns without descending. On some secure circuits this one test does almost all of the work, some branches never split, because their union already discharges.

However this discharge may fail, and it is worth seeing why the algorithm cannot stop here on failure. The union U is typically far larger than d, and a large tuple can be secret-*dependent* while every one of its size-d sub-tuples is secret-independent, hence failing to discharge U says nothing about the individual probes. When the union fails, we descend. We pick a representative $\mathbf{w} \in [\![wl]\!]$ — one probe of the target size, c_j wires from each factor — certify it with Component I, and, if it holds, grow it with Component III (Section 5.4) into a *safe set*

$$\mathsf{safe} \supseteq \mathbf{w}, \qquad \text{every subset of which is secret-independent.}$$

This will be further explained in Component III. Put briefly, one coupling blinds every wire of safe simultaneously, so any subset of safe, of any size, is secret-independent. Because the representative draws at least one wire from every factor, safe meets every W_j. This single fact is what makes the split below both exhaustive and strictly progressing.

5.3.3 Completeness via Vandermonde's Identity

With a safe set in hand the split is forced by a counting identity. Fix safe and cut each factor's ground set along it,

$$W_j = A_j \uplus B_j, \qquad A_j = W_j \cap \mathsf{safe}, \quad B_j = W_j \setminus \mathsf{safe}.$$

Lemma 5.4 (Set Vandermonde)**.** *For disjoint wire sets A, B and any $c \geq 0$,*

$$\binom{A \uplus B}{c} = \biguplus_{i=0}^{c} \left\{ S_A \cup S_B \,:\, S_A \in \tbinom{A}{i},\ S_B \in \tbinom{B}{c-i} \right\},$$

a disjoint *union. Each c-subset $S \in \binom{A\uplus B}{c}$ lies in exactly the block $i = |S \cap A|$. Counting both sides recovers Vandermonde's identity, $\binom{|A|+|B|}{c} = \sum_{i=0}^{c} \binom{|A|}{i}\binom{|B|}{c-i}$.*

The lemma is immediate, a c-subset is determined by its safe part and its unsafe part, chosen independently. Stating it set-wise, and not merely as the numeric identity, is the trick. It says not only how many probes each block holds but exactly which. Applying it in every factor and multiplying out,

$$[\![wl]\!] = \prod_{j=1}^{m} \binom{W_j}{c_j} = \biguplus_{(i_1,\ldots,i_m)} [\![wl_{(i_j)}]\!], \qquad wl_{(i_j)} = \big\langle (i_j, A_j),\ (c_j - i_j, B_j) \big\rangle_{j=1}^{m},$$

one *cell* for each distribution $(i_1, \ldots, i_m) \in \prod_j \{0, \ldots, c_j\}$ recording, factor by factor, how many of a probe's wires come from the safe side. The cells are disjoint and exhaust $[\![wl]\!]$, since every probe declares in each factor exactly how much of it is safe and each cell is again the denotation of a worklist, on which the traversal recurses directly.

One cell needs no recursion however. The all-safe cell $(i_j)_j = (c_j)_j$ collects the probes drawn entirely from the safe side $A_j \subseteq \mathsf{safe}$ and every one of those is secret-independent by the guarantee of Component III. The algorithm skips it and recurses on the rest. Skipping that particular cell means, with each recursion we are strictly decreasing the number of probes we have to consider, which gives us our termination guarantee.

Theorem 5.1 (Soundness of the traversal)**.** CheckAll(wl) *(Algorithm 2) terminates, and if it returns* Secure *then every probe in* $[\![wl]\!]$ *is secret-independent. In particular* CheckAll$\big\langle(d, \mathcal{W})\big\rangle$*, returning* Secure*, certifies* d*-probing security.*

Proof. Termination. Order worklists by the multiset $\{|W_1|, \ldots, |W_m|\}$ of their ground-set sizes, under the multiset order. Take any cell the algorithm recurses on: it is neither the all-safe cell nor empty, so some factor j^* has $i_{j^*} < c_{j^*}$, forcing $B_{j^*} \neq \emptyset$; and $A_{j^*} \supseteq \mathbf{w} \cap W_{j^*} \neq \emptyset$ because the representative meets every factor. So W_{j^*} is replaced by the two strictly smaller ground sets A_{j^*}, B_{j^*}, while no other factor's ground set grows $(A_j, B_j \subseteq W_j)$. The multiset strictly decreases, so the recursion is well-founded.

Soundness, by induction along that order. CheckAll returns Secure only in one of three ways. If the union $U = \bigcup_j W_j$ discharges, every subset of U (hence all of $[\![wl]\!]$) is secret-independent by marginalisation. If a degenerate worklist occurs, $[\![wl]\!]$ is empty or it is the empty probe, which is trivially secure. Otherwise the representative certifies and the recursive cells all return Secure. By Lemma 5.4 those cells together with the skipped all-safe cell partition $[\![wl]\!]$; the all-safe cell is secret-independent by Component III, and each recursive cell is $[\![wl_{(i_j)}]\!]$ for a worklist strictly smaller in the multiset order above, hence secret-independent by the induction hypothesis. Every probe of $[\![wl]\!]$ lies in exactly one part, so all are secret-independent. Finally, since $[\![\langle(d, \mathcal{W})\rangle]\!] = \binom{\mathcal{W}}{d}$, a Secure verdict on $\langle(d, \mathcal{W})\rangle$ certifies that every d-subset of $\mathcal{W}$ is secret-independent, i.e. d-probing secure. □

We close this section with two remarks. The soundness above is one-directional by design: a Secure verdict is trustworthy, but the traversal inherits Component I's incompleteness verbatim. An Insecure verdict is raised only when Component

I declines a representative (Section 5.2.3), never because a subset slipped through unexamined. Lemma 5.4 guarantees the cells cover $[\![wl]\!]$ exactly, so the sweep is genuinely exhaustive. The reported witness is therefore a true counterexample precisely when Component I is complete on it, but in genreal Component I is not complete, so a spurious Insecure can happen. Crucially such an Insecure answer enters through Component I, not through this traversal. Second, the efficiency of the whole scheme rests on the safe set being large: the bigger safe, the fewer and smaller the surviving cells, and the closer the union prune comes to settling a branch outright. That is exactly the quantity Component III's coupling extension is built to maximise, and it is where the approach of this thesis departs from a plain set-splitting search.

5.4 Component III: Extending Probe Sets

Component II leans on a technique which we are yet to discuss. Every time it certifies a representative probe $\mathbf{w}$, it asks for a whole *safe set* of wires around $\mathbf{w}$, every subset of which is again secret-independent, and it uses that set to skip the all-safe cell of its Vandermonde split (Section 5.3.3). Component III is all about that safe set. It takes the certified probe and grows it into such a safe set, reusing the work the rewrite engine has already spent on $\mathbf{w}$ instead of certifying each new wire from nothing. The larger the safe set it hands back, the more the traversal prunes, so the whole scheme is only as efficient as this component is generous.

The three mechanisms we describe share one shape. Each of them walks over the wires of the circuit that are not yet in the safe set and decides, for a candidate u, whether the set stays jointly secret-independent once u is added. What separates them is the certificate they reuse and the test they run on it. Their soundness rests on a single fact which is the following: suppose φ is a bijection of the randomness space, and suppose that for every wire u in a set U the composite $u \circ \varphi$ is independent of the secrets. Here u is read as the wire function of Definition 2.5, so $u \circ \varphi$ is the map $\rho \mapsto u(\varphi(\rho); \mathbf{a})$; when we call it *secret-free* below we mean the expression obtained by applying the recorded substitutions to $\mathrm{expr}(u)$, and Lemma 5.5 is what makes that syntactic test agree with the semantic condition stated here. Since φ preserves the uniform law, the joint distribution of the wires of U agrees with the joint distribution of the composites $u \circ \varphi$, and the latter does not move with the secrets. So the joint law of U does not vary with the secrets, and by marginalisation neither does the law of any subset of U. A single bijection that blinds every wire of U at once thus

certifies the entire set. We should add that the certified probe $\mathbf{w}$ is always kept in the safe set without a fresh test of course; the per-wire tests below are run only on the other candidates. We present the three mechanisms from the weakest to the strongest and adopt the last.

5.4.1 Closure-Driven Extension

The first mechanism is set apart from the other two by *when* it runs. A closure is not computed after a probe has been certified. It is grown together with the probe, in the same topological pass that assembles it. As the traversal walks the circuit and picks wires to fill the chosen probe of the current worklist, it keeps a running set of the wires already forced by the ones picked so far, and each new choice drains that set forward. Two local rules drive the drain. A gate all of whose inputs are already known becomes known in turn, since its value is then fixed, and when a known wire is an $\oplus$ of inputs of which all but one are known, that last input becomes known as well, recovered as the sum of the wire and the others. The pass finishes with the chosen probe and, around it, every wire these two rules could deduce from it.

Everything the drain adds is a deterministic function of the probe. Writing $\mathbf{w}$ for the chosen probe and u for a deduced wire, we have $u = f(\mathbf{w})$ for the function f that the rules compose, so u carries no randomness the probe does not already carry, i.e. $H(u \mid \mathbf{w}) = 0$. Adjoining it leaves the joint entropy untouched, $H(\mathbf{w} \cup \{u\}) = H(\mathbf{w})$, and the pair $(\mathbf{w}, u)$ is a deterministic image of $\mathbf{w}$. Its dependence on the secrets is therefore exactly the probe's, which the engine has already certified to be none. That is the whole of the closure argument, and it needs no coupling.

What the closure buys is little. In a masked circuit almost no wire is a plain function of a handful of others, so the set deduced around a small probe is rarely more than a few relabellings, and the traversal is left to split nearly the whole space by hand. The shortfall sharpens as Component II recurses. Its worklist factors draw from wire sets carved out of the safe and unsafe splits, so deep in the recursion the candidate pool is a scattering of wires gathered from distant, unrelated parts of the circuit. Closure deduces a wire only from its immediate neighbours, and those neighbours become rare in a fragmented pool, so the deduction finds little to do exactly where the traversal needs the help most. In our experiments this mechanism extended probes the least of the three, although it is the cheapest of them, since the deduction rides along inside a pass the traversal was making anyway and never calls the engine a

second time. Its worth is mostly conceptual.

5.4.2 Replay-Driven Extension

A stronger idea, due to Barthe et al. (Barthe et al., 2019), is to reuse the derivation rather than the probe. Certifying $\mathbf{w}$ left a short record of the steps that discharged it (Section 5.2.6), and each step is an operation one can carry out on any expression, not only on $\mathbf{w}$. Replay-driven extension takes a candidate wire u from the pool of the current worklist and runs that same record on it, step by step. If u comes out free of secrets at the end, then it is blinded by the very derivation that blinded the probe, and it joins the safe set. This reaches well past the closure, because a wire no longer has to be a function of the probe to survive the replay. It only has to be carried to a secret-free form by the same steps, which many of the sibling wires of a masked gadget are.

Barthe et al.'s derivations are built from two kinds of step. The first is the substitution of an invertible random by its context, the optimistic-sampling move our own engine makes. The second is an algebraic normalisation, which rewrites an expression into its algebraic normal form (its unique multilinear $\mathbb{F}_2$ polynomial, Section 2.1.1) and so lets products cancel. Barthe turns to it only when no substitution applies, and records it in the derivation like any other step. When the replay meets such a step it normalises the candidate too.

For us replays are a little problematic. Our engine does not lean on normalisation to free a buried random. It factors the expression instead (Section 6.1.3), reshaping a product so that a random trapped inside it moves into an additive position where a substitution can reach it. Factoring wins the engine circuits that substitution and normalisation alone would miss, but it is not an easily replayble step. Re-applying a factoring to an unrelated candidate wire does not carry over the blinding the way re-applying a substitution does, so the record we keep holds only the substitutions and leaves the factoring implicit. Replaying that record on a candidate is then incomplete. A wire the derivation genuinely blinds can fail it, because the cancellation that clears its secret was arranged by a factoring the replay never performs. That mismatch is the whole reason why we take one further step, and view the derivation as a coupling function.

5.4.3 Coupling-Driven Extension

The mechanism we adopt keeps replay's use of the derivation and replaces its final test with an exact one. Section 5.2.6 already showed how to view the recorded derivation as a single bijection φ of the randomness space. Coupling-driven extension admits a candidate wire u exactly when $u \circ \varphi$ is independent of the secrets, and it decides that independence semantically rather than by surface syntax. Every wire replay admits is admitted here too, and so are the wires replay had to withhold because a factoring had to set up cancellations.

The candidates are not all the wires of the circuit. They are the wires of the current worklist, the pool C that Component II is trying to certify (Section 5.3), and the probe $\mathbf{w}$ is kept among them with no test at all. To decide $u \circ \varphi$ is secret-free exactly, for every candidate, is not cheap, so the work is staged. Initially, a fast probabilistic filter throws out the candidate wires that are incompatible with the coupling function we derived via evaluations.

Algorithm 3 Coupling-driven extension (Component III)

function EXTEND($\mathbf{w}, \varphi, C$)
Require: certified probe $\mathbf{w}$, coupling φ, candidate pool C from the worklist
Ensure: safe set safe with $\mathbf{w} \subseteq \mathsf{safe} \subseteq C$, with every subset secret-independent
    $\mathsf{safe} \leftarrow \mathbf{w}$
    for all $u \in C \setminus \mathbf{w}$ **do**
        if FASTREJECT(u, φ) **then**
            continue ▷ Phase 1: probabilistic filter, rejects proven-dependent u only
        end if
        if SURFACEFREE(u, φ) **or** ANFFREE(u, φ) **then** ▷ Phases 2–3: exact, decides $u \circ \varphi$ secret-free
            $\mathsf{safe} \leftarrow \mathsf{safe} \cup \{u\}$
        end if
    end for
    return safe
end function

The fast filter is an evaluation under two runs. It fixes random values for the randoms, evaluates $u \circ \varphi$ once with the secrets also drawn at random and once with the secrets set to zero, and compares. If the two outcomes disagree, then $u \circ \varphi$ genuinely moves with the secrets, and u is dropped. Agreement proves nothing on its own, since a dependence can hide from any one tape, so a survivor is not yet admitted. Running the trial over many tapes makes a surviving dependence less likely but never impossible, which is why the filter is allowed to reject but never to

accept.

Then the derivation is composed with u, and the result is first read by ordinary $\mathbb{F}_2$ cancellation, which is cheap and clears the common case. When a secret still shows, the wire is converted to its algebraic normal form and admitted exactly when no secret variable survives there. The probabilistic filter and this exact stage differ in kind, so it is worth recording that together they never admit a leaking wire and never turn away a blinded one.

Lemma 5.5 (The extension test is exact)**.** *EXTEND admits a candidate u if and only if $u \circ \varphi$ is independent of the secrets.*

Proof. Write $v = u \circ \varphi$. Its algebraic normal form (ANF) is the unique multilinear $\mathbb{F}_2$ polynomial computing v, so a variable is absent from that form exactly when v does not depend on it. In particular,

$$v \text{ is independent of the secrets} \iff \text{no secret variable occurs in the ANF of } v.$$

A wire is admitted only at the exact stage. Phase 2 accepts when the surface form of v carries no secret, and Phase 3 accepts when the ANF of v carries no secret variable. By the equivalence, each makes v independent of the secrets, so every admitted wire is truly blinded. Conversely, suppose v is independent of the secrets. With the randoms fixed, v is then constant in the secrets, so the filter's two evaluations agree on every tape and never reject u, and it reaches the exact stage. And because the ANF of v must not carry a secret variable, Phase 3 will admit it. Hence u is admitted if and only if v is independent of the secrets. □

Theorem 5.2 (Safe set)**.** *Let* $\mathsf{safe} = \textsc{Extend}(\mathbf{w}, \varphi, C)$ *for a probe* $\mathbf{w}$ *certified by Component I with coupling φ. Then the joint distribution of the wires of* safe*, over a uniformly random ρ, does not depend on the secret assignment, and in particular every subset of* safe *is secret-independent.*

Proof. By Lemma 5.5, safe consists of the probe $\mathbf{w}$ and the candidates u whose composite $u \circ \varphi$ is independent of the secrets. Since that composite does not depend on the secret assignment, it is a function of the randomness alone; so simply write its value as $\overline{u}(\rho)$.

Fix a secret assignment $\mathbf{a}$ and let $\varphi_{\mathbf{a}}$ be the induced bijection of the randomness space (Section 5.2.6). Chaining Lemma 2.1 through the recorded steps gives, for

every $u \in \mathsf{safe}$ and every ρ,

$$u\Big(\varphi_{\mathbf{a}}(\rho); \mathbf{a}\Big) \;=\; (u \circ \varphi)(\rho; \mathbf{a}) \;=\; \overline{u}(\rho).$$

The left-hand side is u evaluated at the relabelled randomness assignment $\varphi_{\mathbf{a}}(\rho)$ under the secrets $\mathbf{a}$, so it is a bit value; the first equality is the substitution lemma, the second one comes from the admission condition. Both copies of $\mathbf{a}$, the one inside $\varphi_{\mathbf{a}}$ and the one feeding u its secret inputs, cancel, and the value depends on ρ alone.

Now hold $\mathbf{a}$ fixed and draw ρ uniformly. As $\varphi_{\mathbf{a}}$ preserves the uniform law, $\varphi_{\mathbf{a}}(\rho)$ is uniform too, so the joint law of the safe wires under $\mathbf{a}$, that of $\Big(u(\rho; \mathbf{a})\Big)_{u \in \mathsf{safe}}$, equals the joint law of $\Big(u(\varphi_{\mathbf{a}}(\rho); \mathbf{a})\Big)_u = \Big(\overline{u}(\rho)\Big)_u$. This last family depends on no secret, so the joint law is one and the same for every $\mathbf{a}$. Hence the distribution of safe does not depend on the secret assignment, nor any of its subsets. □

Finally we observe that the other two mechanisms admit nothing if this one does not.

Proposition 5.1 (Closure and replay sit inside the coupling extension)**.** *Fix a certified probe* $\mathbf{w}$ *with coupling* φ*, and write* $\mathsf{safe}_{\mathrm{clo}}$*,* $\mathsf{safe}_{\mathrm{rep}}$*, and* $\mathsf{safe}_{\mathrm{cou}}$ *for the wires admitted by the closure, replay, and coupling extensions. Then* $\mathsf{safe}_{\mathrm{clo}} \subseteq \mathsf{safe}_{\mathrm{cou}}$ *and* $\mathsf{safe}_{\mathrm{rep}} \subseteq \mathsf{safe}_{\mathrm{cou}}$.

Proof. For replay, a wire is admitted only after its replay rechecks to a secret-free form, which means $u \circ \varphi$ is independent of the secrets; the coupling test admits every such wire by Lemma 5.5. For the closure, an admitted wire is a gate-composition $u = f(\mathbf{w})$. Evaluation respects that composition, so $u \circ \varphi = f(\mathbf{w} \circ \varphi)$ as functions of the randomness; the coupling leaves $\mathbf{w} \circ \varphi$ independent of the secrets, and a gate-composition of functions independent of the secrets is again independent of them, so $u \circ \varphi$ is too and the coupling test admits u. □

6. IMPLEMENTATION OF COPPULA

6.1 Expression Representation

The rewrite engine of Component I (Section 5.2) works entirely on the expressions of a probe. It looks for a random in an additive context, substitutes it by its context, and asks whether any secret is still in sight. Every one of these moves is an operation on the *form* of the expression, so the structure that holds that form is what decides how cheap the engine can be. This section describes that structure. It is a hash-consed n-ary DAG over $\mathbb{F}_2$, wrapped in a layer of smart constructors that keep every expression canonical, and extended with a multiplicative factoring pass that reshapes products so the engine can reach randoms otherwise buried inside them.

Remember that the circuit of Definition 2.3, once the gate set is fixed to $\{\oplus, \odot\}$ (Section 2.1.3), is strictly 2-ary, and that is deliberate. Each 2-input gate output is a distinct observable, so collapsing gates would hide intermediate wires the adversary is allowed to probe (Section 2.3.2). The representation here is not the circuit, and this is important. It is how we store the expression $\mathrm{expr}(w)$ of a wire (Definition 2.4) for the algebra, and there flattening a chain of same-typed gates into one n-ary node loses nothing, since only the wire's function matters and not its gate-by-gate history. So the two structures (one faithful to the graph and one keeping the expressions) live side by side, exactly as the layering of Section 2.1.3 prescribes. The 2-ary circuit graph stays the authority on what may be probed, and the DAG is the authority on what each probed wire computes.

6.1.1 A Hash-Consed n-Ary Circuit DAG

An expression is a node of one of four kinds. It is a *leaf* carrying an input variable (secret, random, or public), a *constant* 0 or 1, an n-ary sum $\oplus$ over a list of operand nodes, or an n-ary product $\odot$ over a list of operand nodes. This is the expression grammar of Section 2.1.3 with the two operators taken n-ary rather than binary, as flattening (Section 6.1.2) kills the binary form. A wire is stored as the node its expression builds, and the whole circuit is a single DAG in which wires that share sub-structure share nodes. Building it is the recursion of Definition 2.4 run once, in topological order, with the sharing described next keeping it from unfolding into a tree.

Sharing is the first reason for the design, and it is not incidental. A masked gadget reuses the same handful of masks and shares across many gates, so the naive syntax tree of a probe would carry the same subexpression over and over again. We store each distinct node once. A table, keyed by a node's kind together with its operand list, which maps a structure to the identifier of the node already built for it; this is *hash-consing*. Constructing a node is then a lookup in that table. On a hit we return the existing identifier, and on a miss we allocate a fresh one and record it. Two expressions that are structurally the same are therefore the same node, physically, however many times they are written. One can think of it as a lightweight e-graph.

The second reason is equality. Because interning makes structural identity coincide with identifier identity, deciding whether two expressions are equal is a comparison of two integers, with no traversal at all. The rewrite engine leans on this everywhere. Recognising that a substitution has made two operands coincide so that they cancel, checking whether a rewritten probe matches one already seen, and, in Component III, testing a candidate wire's coupling image against a secret-free target, are each an $O(1)$ test on identifiers rather than a walk over two trees. We should note that this fast equality is only as meaningful as the expressions are canonical: hash-consing alone shares expressions written identically, and it is the next subsection that makes identical identifiers mean equal values. We also keep the identifiers of the random and secret leaves in their own lists, so iterating over vars($\mathbf{w}$), or scanning for a surviving secret, never has to walk the entire DAG.

6.1.2 Canonicalization: Flattening and Cancellation

Hash-consing shares expressions that are written identically, which on its own is weak. In $\mathbb{F}_2$ the sums $a \oplus b$ and $b \oplus a$ are equal, yet written in that order they are two different nodes. We close this gap with a layer of *smart constructors*. Nothing builds a sum or a product node directly; every sum is made through one constructor and every product through another, and these put the operands into a canonical form before interning. Two expressions with the same value then reach the same canonical node, so the fast equality of Section 6.1.1 becomes equality of value and not merely of syntax, at least for the identities we fold in, which are exactly the ones the engine needs.

Three normalisations do the work. The first is *flattening*. Associativity lets a sum of sums be one flat sum, and likewise a product of products be one flat product. This is where the n-ary shape earns itself, as a chain of 2-ary XOR gates from the circuit graph collapses into a single sum node holding all the leaves, and any nesting is quotiented away. The second is *ordering*. Commutativity lets us keep each operand list sorted by identifier, so the order of writing is irrelevant to identity and $a \oplus b$ and $b \oplus a$ intern to one node. The third folds in the $\mathbb{F}_2$ identities the engine's cancellations depend on. In a sum, the constant 0 is dropped as the additive identity, and a repeated operand is deleted in pairs, since $x \oplus x = 0$. In a product, the constant 1 is dropped as the multiplicative identity, the constant 0 collapses the whole product to 0 by annihilation, and a repeated operand is kept only once, since $x \odot x = x$ by idempotence (Section 2.1.1). Degenerate arities settle the same way. An empty sum is 0, an empty product is 1, and a one-operand sum or product is that operand.

The payoff is that the one algebraic move the whole rewrite method turns on happens for free. When the engine substitutes a single-occurrence random r by its context, replacing the parent $x = e \oplus r$, the cancellation $e \oplus (e \oplus r) = r$ that Corollary 2.1 promises is not a step the engine performs. It re-interns the rewritten expression through the sum constructor, the two copies of e meet in the sorted operand list and annihilate, and the canonical result is already the collapsed form. The idempotent and annihilating collapses that a general substitution triggers happen the same way. The engine proposes rewrites, and the constructors do the algebra.

6.1.3 Multiplicative Factoring

Canonicalisation normalises the additive structure of an expression, but it leaves products opaque, and Section 5.2.4 discussed why that matters. The find-rules see a random only in additive position, so one buried inside products is beyond their reach. Multiplicative factoring is the pass that surfaces it. Here we give the details regarding that transformation.

6.1.3.1 The Factoring Transformation

Factoring applies the distributive law right to left, over a sum whose operands are products. If a node f appears as an operand in two or more of those products, we pull it out,

$$(f \odot a) \oplus (f \odot b) \oplus \mathit{rest} = \Big(f \odot (a \oplus b)\Big) \oplus \mathit{rest},$$

where *rest* are the summands not carrying f, and a, b are what remains of the two products once f is removed. The identity holds over $\mathbb{F}_2$, so only the form changes and not the function denoted (Lemma 5.1).

What we pull out is chosen greedily. Over the operands of a sum we count, for each candidate factor, in how many of the product summands it appears, and take the one with the highest count, provided it appears at least twice, since a factor sitting in a single product buys nothing. Ties are broken by identifier, for determinism. With the factor f fixed, we partition the summands into those carrying f and those not, strip f from each of the former, sum the remainders into an inner sum, multiply that by f, and add the untouched summands back, every construction going through the canonicalising constructors of Section 6.1.2. Factoring is then applied recursively, bottom up over the DAG and repeated on its own result, until no sum has a factor of multiplicity two or more.

For a concrete instance we take a probe from the first-order masked quadratic bijection Q_{12}^4, one of the gadgets we verify. Its inputs are shared as $a_1 = a \oplus r_0$, $b_1 = b \oplus r_1$, $c_1 = c \oplus r_2$ together with the mask shares $b_2 = r_1$ and $c_2 = r_2$, where a, b, c are secret and r_0, r_1, r_2 are fresh randoms. One of the gadget's recombination wires gathers five product and share terms, and after canonicalisation its expression is

$$w = (a \oplus r_0) \odot (b \oplus r_1) \oplus (a \oplus r_0) \odot (c \oplus r_2) \oplus (c \oplus r_2) \oplus (a \oplus r_0) \odot r_1 \oplus (a \oplus r_0) \odot r_2.$$

As it stands the simple rule cannot finish. It clears r_0, whose only occurrence is the additive one in $a \oplus r_0$, but b and c then remain, and neither r_1 nor r_2 is a single-occurrence random. Each shows up both additively, in a share, and again inside the products, so the uniqueness the simple rule needs is absent and the general rule's side condition fails as well. Factoring is what breaks the deadlock. The factor $a \oplus r_0$ is shared by four of the five summands, and pulling it out drops their cofactors into a single inner sum,

$$w = (a \oplus r_0) \odot \Big[(b \oplus r_1) \oplus (c \oplus r_2) \oplus r_1 \oplus r_2\Big] \oplus (c \oplus r_2) = (a \oplus r_0) \odot (b \oplus c) \oplus c \oplus r_2,$$

where inside the bracket $r_1 \oplus r_1$ and $r_2 \oplus r_2$ cancel by the additive law of Section 6.1.2. The surplus copies of r_2 go with them, and what is left carries r_2 additively and exactly once. Now the single-occurrence pattern of Section 5.2.2 applies. The engine relabels the whole expression as a fresh random and reads w as secret-free.

6.1.3.2 Enlarging the Class of Verifiable Circuits

Factoring is sound by Lemma 5.1, so it never lets an insecure probe pass. What it changes is completeness, and it strictly enlarges the class of probes the engine can certify. Every probe the engine discharges without factoring it still discharges, since Algorithm 1 tries the unfactored form first and factors only on failure. And some probes, like the one above, are discharged only once factoring has exposed a random no additive rule could otherwise reach. The verifiable class with factoring is therefore a strict superset of the class without it.

Two cautions temper the picture, both settled already in Chapter 5. First, factoring is not monotone in the cheap direction. It can just as well destroy a single-occurrence opportunity that the unfactored form exposed, which is exactly why the engine tries the simple rule unfactored before it factors, rather than factoring once up front (Section 5.2.2). It has to be interleaved with substitution, not run as a preprocessing step. Second, factoring is not a replayable operation. Re-applying it to an unrelated candidate wire does not carry over the blinding the way re-applying a substitution does, and this is what forces Component III to read a derivation as a coupling with an exact endpoint test rather than as a script to rerun (Section 5.4.3). The gain in verifiable circuits is real, but it is bought with the extra machinery those sections build around it.

6.2 Implementing the Rewrite Engine

Algorithm 1 presents Component I as a single tiered procedure, but the tool realises its rule set twice, and the reason for the two is worth an explanation. One is a cheap, verdict-only path that answers secret-independence without ever rewriting the DAG, and the other is a complete path that rewrites for real and hands back the coupling. The complete path tries the cheap rewrites first on the unfactored probe, then once more after factoring, and only then falls through to the general rewrite loop. The two paths have different callers. Component II's large-set pruning (Section 5.3.2) throws the big union at the engine and wants only a yes or no, as fast as possible, whereas a certified representative must come back with the coupling that Component III will use (Section 5.4.3). The next two subsections take the paths in turn.

6.2.1 Reference-Counted Single-Occurrence Discharge

This path serves the call for the large-set prune of Section 5.3.2, which hands the engine the union $U = W_1 \cup \cdots \cup W_m$ of the current worklist and wants nothing back but a verdict. The union is typically far larger than d, it is discharged on many branches, and its answer is never extended, so the whole cost of building a derivation would be wasted on it. Representatives of a worklist do not come here; they are sent straight to the complete loop of Section 6.2.2.

What is being implemented is SimpleRewrite of Algorithm 1. Recall its shape. It collects into *todo* the randoms of vars($\mathbf{w}$) that occur exactly once, additively, and are not probe roots; it orders them by multiplicative depth; and it repeatedly pops one such r, takes its unique additive parent $x = e \oplus r$, marks x as a fresh random leaf, records the step, and drops r — which may expose a new single-occurrence random and so refill *todo*. It stops when the probe is secret-free or no candidate is left.

The observation the implementation exploits is that this rule needs very little about a random: how many times it occurs, whether that occurrence is additive, and which parent to splice away. None of that requires rewriting anything. So a single depth-first pass over the probe's sub-DAG records, for every node, how many parents it

has, how many of those are sums, and its multiplicative depth, and the rule is then run against those counts. A random is a candidate exactly when it has one parent, that parent is a sum, and it is not a probe root, which is *todo* read off the counts. Firing the rule is a local edit to the counts and not to the DAG: mark the additive parent as freshly random, drop the random, and decrement the parent counts of the parent's other children. A sibling whose count thereby falls to a single additive occurrence re-enters the *todo* list. The probe is secure once no secret is reachable from any root, which is the SecretFree test.

Keeping observed wires out of the todo list is an important soundness condition. A random that is itself a probe root must never be relabelled, since its value is exactly what the adversary sees, and marking its parent fresh while conditioning on it would be mathematically unsound. The cascade re-inserts a candidate only after the same root check.

The one thing the path drops is the record. Its cascade steps fire on already-relabelled parents rather than on tape variables, so they are not the leaf relabels that Section 5.2.6 assembles a coupling from, and replaying them against another wire would diverge from the certified derivation. There is therefore no T here and no coupling. This does not contradict Lemma 5.2, which speaks of SimpleRewrite as Algorithm 1 states it, pushing every step onto T; what this path implements is that rule's verdict alone. Anyways a verdict is all the caller needs.

6.2.2 Recomputing the Find-Rules per Round

The complete loop (GeneralRewrite, Section 5.2.3) substitutes for real, as in it alters the DAG. It picks a random and an additive occurrence $x = e \oplus r$ and rewrites the probe by $r \mapsto e \oplus r$ throughout, re-interning the result through the canonicalising constructors of Section 6.1.2, which is what makes the chosen occurrence collapse to a bare r while every other absorbs e. Because the substitution genuinely reshapes the expression, the incremental counts of Section 6.2.1 cannot be carried across it. So the loop recomputes them. Each round runs one fresh depth-first pass over the current form, and that single pass does double duty. It answers whether any secret still survives, which is the loop's success test, and it produces the parent counts and depths the find-rules need. The pass is restricted to the randoms actually present in the probe rather than every random in the circuit.

On that snapshot, the loop first tries FindSimple, the same single-occurrence additive pattern as before, since a general substitution can turn a twice-occurring random into a once-occurring one; failing that, FindGeneral looks for a not-yet-used random with an additive parent whose other children do not mention it, which is the side condition Lemma 2.1 needs. That last check is a reachability question, does r occur anywhere below the context e, answered by a memoised walk whose memo is shared across the candidate's parents so the sub-DAG is not re-explored from scratch. Both rules prefer the shallowest candidate. When both stall the loop factors once and retries (Section 6.1.3), and a fuel counter backstops the whole thing, as Section 5.2.3 discussed.

Every substitution the loop commits is pushed as a pair $(r, e \oplus r)$, so what accumulates is exactly the record T of Section 5.2.1. The loop hands it back alongside its verdict, and it is the derivation Component III replays.

6.3 Evaluating the Coupling

Component III decides, for each candidate wire u, whether $u \circ \varphi$ is free of secrets, where φ is the coupling the engine just recorded and $u \circ \varphi$ is read as in Section 5.4. Algorithm 3 gives that decision, together with a probabilistic filter first and an exact test after. Both stages need to evaluate a wire through the coupling, and this section describes how that evaluation is done cheaply.

6.3.1 Bit-Sliced Evaluation Through a Coupled Environment

The filter compares two runs of $u \circ \varphi$, one with the secrets drawn at random and one with them zeroed, and rejects u when they disagree (Section 5.4.3). A single pair of runs proves little, so the filter wants many, and the representation makes many nearly as cheap as one. Every wire value is carried as a machine word whose 64 bits are 64 independent $\mathbb{F}_2$ evaluations, one per lane. A sum is a word exclusive-or and a product a word and, so one traversal of the sub-DAG evaluates all 64 lanes at once, and node values are cached across the traversal so a shared subexpression is computed a single time.

Evaluating $u \circ \varphi$ never rebuilds u. Recall the coupling is a list of relabels $r \mapsto e \oplus r$; rather than substitute them into u, we precompute a *coupled environment* that binds each random to the value of its net image under the whole list and then evaluate u against it. Since a relabel's image may mention randoms that later relabels move, the environment is built by folding the list from right to left, so that by the time a random is bound, every relabel acting on its image is already computed. Evaluating u against that environment then agrees with evaluating $u \circ \varphi$ against the original one, so the two environments, secrets-as-random and secrets-as-zeros, are each built once and reused for every candidate, with their evaluation caches shared across.

6.3.2 From Probabilistic Filter to Exact Verdict

Agreement across lanes is solid evidence but not a proof, so the filter is allowed to reject but never to accept (Lemma 5.5). A survivor has to be decided exactly. The coupling is applied to all survivors together, sharing one substitution memo per step rather than replaying the whole list per candidate, and each image is then read in two ways. First by ordinary $\mathbb{F}_2$ cancellation over the canonical form, which is cheap and clears the common case. When a secret still shows there, the image is converted to its algebraic normal form, and the wire is admitted exactly when no secret variable occurs in its ANF. The ANF is the canonical multilinear form (Section 2.1.1), so a secret is absent from it precisely when the wire does not depend on that secret; this is what makes the test exact rather than merely sound.

7. EVALUATION

7.1 Benchmarks

We evaluate Coppula on a suite of masked gadgets drawn from the masking literature, grouped by family in Table 7.1. The suite collects the standard nonlinear building blocks of masked hardware at masking orders one through four, so that both the secure cases and the expected failure cases are exhibited.

The gadgets can be categorized into 4 large classes. The ISW multiplication and its refresh gadgets are the field-agnostic core of the higher-order masking of Ishai, Sahai, and Wagner (Ishai et al., 2003) and Rivain and Prouff (Rivain & Prouff, 2010). We instantiate them over $\mathbb{F}_2$ at orders one through four, similar to the verification suites of Barthe et al. (Barthe et al., 2015, 2019) do. The domain-oriented masking (DOM) gadgets (Groß et al., 2016), the Trichina AND (Trichina, 2003), and the threshold-implementation (TI) AND (Nikova et al., 2006) are the recurring masked-multiplication cores of the same suites. The threshold implementations of small S-boxes of Bilgin et al. (Bilgin et al., 2015) supply the rest, made up of the quadratic-class representative Q_{12}^4, the Fides/PRIMATEs five-bit S-box, the Keccak χ sharings, and the $x + yz$ and $x + yz + xyz$ constructions.

On top of these standard gadgets we extend one family, the d+1 quadratic S-box family (the Q^3 and Q^4 rows of Table 7.1). These are first-order, randomness-free d+1 maskings with share compression of the quadratic permutation classes of (Bilgin et al., 2015), presenting a low-randomness hardware style, and we include them because they stress the factoring capability of the engine, as Section 7.3.1 mentions.

We should adress a limitation before diving into the methodology. As we know, the model is value-based over $\mathbb{F}_2$, so the GF(2^8) benchmarks of (Barthe et al.,

2015), the AES S-box and rounds, are present only through their field-agnostic ISW-multiplication core. A pair of representative gadgets is transcribed as netlists in Appendix A.

7.2 Methodology and Metrics

Each gadget is verified by the compiled Lean executable, one run per configuration, on an AMD Ryzen 7 5800H. For a gadget and a probing order d we record the verdict, the wall-clock time, and two structural counts that are insensitive to the machine and to engine-version differences. The first is the number of *discharges*, the calls made to the rewrite engine of Component I; the second is the extension yield, or *free wires*, the wires that Component III certifies for nothing beyond the probes actually discharged.

Correctness is checked on two levels. Each hand-transcribed sharing was brute-forced to confirm that its shares recombine to the intended function before it entered the suite. And every configuration is run twice, once with Component III's extension enabled and once against a naive baseline that discharges each probe from scratch with no extension. Of course, the two must return the same verdict, which they do on every entry. Finally, the extension mechanisms of Chapter 5 are compared against one another by porting the whole suite to three engine variants, coupling-driven, witness replay, and closure-driven, and running each under its own mechanism (Table 7.1).

7.3 Results

Figure 7.1 shows verification time against circuit depth for a chain of ℓ cascaded ISW-AND gadgets, each computing $y^{(k)} = y^{(k-1)} \odot x^{(k)}$ with fresh randomness and each secure at every length. The time axis is logarithmic, so the near-straight lines show verification cost growing exponentially in the chain length even though the netlist grows only linearly (8 wires per stage). The coupling engine (solid lines) runs a constant factor faster than the naive, no-extension baseline (dashed lines)

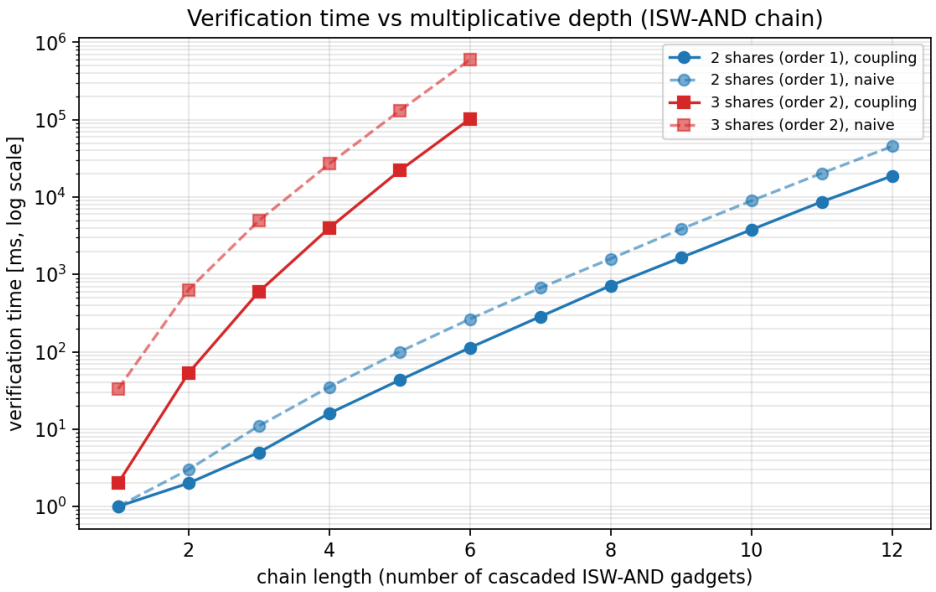

Figure 7.1 Verification time versus circuit depth

but shares its asymptotics. This shows the scalability boundary of the present representation.

7.3.1 Verification Outcomes and Case Studies

Across all forty examples the verdict matches the ground truth. The secure constructions are certified, and the genuinely insecure ones return a witnessing probe, which are: the DOM and TI ANDs probed one order beyond their design, the misassociated Trichina AND, the $x+yz$ TI at order three, and the Q_3^3 and Q_{300}^4 classes. Two case studies are worth focusing on.

- **A first-order TI beyond its order:** The $x+yz$ four-share threshold implementation of Bilgin et al. (Bilgin et al., 2015) is a first-order countermeasure. Coppula certifies it secure at orders one and two and returns a distinguishing probe at order three (Table 7.1). This is the expected behaviour of a first-order TI rather than a flaw in the construction; what it exercises is the tool's counterexample-returning side on a gadget whose failure order is known.
- **The factoring family:** The d+1 quadratic S-box family is where the coupling engine separates from the alternatives. These gadgets interleave their shares without fresh randomness, so a certifying rewrite must first factor a share-sum,

$u_i \odot w_1 \oplus u_i \odot w_2 \mapsto u_i \odot (w_1 \oplus w_2)$, before the blinding random telescopes into view (Section 6.1.3). The factoring-free replay engine, mirroring `maskVerif`, therefore false-reports Insecure on the four quadratic-feedback classes Q_2^3, Q_{12}^4, Q_{293}^4, Q_{299}^4, together with the direct Q_{12}^4 sharing, while the coupling engine, which factors, verifies every one (the † rows of Table 7.1). The two classes the tool reports insecure, Q_3^3 and Q_{300}^4, are exactly the quadratic classes that admit no uniform three-share TI (Bilgin et al., 2015). This family is the concrete example where our engine certifies circuits that a comparable factoring-free tool cannot.

7.3.2 Redundancy in the Search

In our experiments, we have noted that the traversal of Component II, combined with the extensions of Components III, can discharge the same probe more than once. The first source of this redundancy was the large-set prune of Section 5.3.2. Before splitting a batch, Component II discharges the whole union to try the prune, and on a secure circuit the same union recurs on neighbouring branches. After this observation, we memoised the verdict of every set the engine discharges, so such a repeat is served from the cache instead of re-running the engine; the union verdict is all the prune needs anyways, so no coupling has to be recomputed. On the ISW multiplication the cache spares three of these engine runs at three shares, nine at four, and eighty at five, and it grows with order. What remains is a milder redundancy and comes from Component III's extensions. Now and then two different certified probes grow into the same safe set, an *extension collision*, so one region of the observation space is covered twice. These stay few, none at three shares and eighteen at five, and, each carrying its own coupling, they are not memoised away. They are a mild over-count that never threatens soundness.

7.3.3 The Cost and Benefit of Extending Probe Sets

Table 7.1 compares the three extension mechanisms of Chapter 5, coupling-driven (C, Section 5.4.3), witness replay (R, Section 5.4.2), and closure-driven (Cl, Section 5.4.1), across the benchmark suite. Each entry is a C/R/Cl triple giving the verdict (S = Secure, I = Insecure), the wall-clock time, the number of rewrite-engine

discharges, and the extension yield (*free wires*, the wires certified beyond the chosen probes). Replay is factoring-free by design, like `maskVerif`, so it false-reports Insecure on the five factoring-dependent rows marked with a †, where the coupling and closure engines, which keep factoring, agree.

Table 7.1 Extension-mechanism comparison across the benchmark suite.

Circuit	d	Coupling / Replay / Closure			
		Verdict	Time [ms]	Discharges	Free wires
ISW multiplication (Ishai et al., 2003)					
2 sh.	1	S/S/S	0/0/1	7/7/15	2/2/0
3 sh.	2	S/S/S	3/1/34	34/34/309	31/31/8
4 sh.	3	S/S/S	46/37/3190	338/338/10770	437/437/439
5 sh.	4	S/S/S	1125/845/409472	4735/4735/632834	8078/8078/37513
DOM AND (Groß et al., 2016)					
2 sh.	1	S/S/S	1/0/0	11/11/15	2/2/0
2 sh.	2	I/I/I	0/0/1	2/2/6	0/0/2
3 sh.	2	S/S/S	5/4/35	88/88/325	44/44/7
4 sh.	3	S/S/S	104/90/3419	859/859/13291	784/784/532
Trichina AND (Trichina, 2003)					
well-formed	1	S/S/S	0/1/1	11/11/15	2/2/0
mis-associated	1	I/I/I	1/0/1	10/10/12	1/1/0
TI AND (Nikova et al., 2006)					
3 sh.	1	S/S/S	0/0/1	9/9/27	9/9/0
3 sh.	2	I/I/I	1/1/1	10/10/10	5/5/2
Mask refreshing (Ishai et al., 2003; Rivain & Prouff, 2010)					
additive, 3 sh.	2	S/S/S	0/0/0	6/6/8	1/1/0
additive, 4 sh.	3	S/S/S	1/0/1	11/11/17	6/6/0
ISW, 3 sh.	2	S/S/S	0/0/0	6/6/10	4/4/0
ISW, 4 sh.	3	S/S/S	0/1/2	12/12/51	24/24/0
Keccak χ (Bilgin et al., 2015; Groß et al., 2016)					
DOM, 2 sh.	1	S/S/S	14/11/29	49/49/107	29/29/5
DOM, 3 sh.	2	S/S/S	2807/1797/11563	2865/2865/14047	2671/2671/645
TI, 3 sh.	1	S/S/S	21/14/103	43/43/183	68/68/5
TI, 3 sh.	2	I/I/I	21/14/72	57/57/352	92/92/27
TI S-boxes and quadratic sharings (Bilgin et al., 2015)					
Q_{12}^4 direct, 2 sh.	1	S/I†/S	2/2/5	27/26/47	16/16/10
Q_{12}^4 TI, 3 sh.	1	S/S/S	5/4/25	25/25/83	30/30/6
Fides AB1, 4 sh.	1	S/S/S	1941/972/36461	55/55/1127	537/537/14
$x+yz$ direct, 3 sh.	1	S/S/S	1/0/3	7/7/29	10/10/0
$x+yz$ w/ CT, 3 sh.	1	S/S/S	1/1/3	11/11/29	9/9/0
$x+yz$, 4 sh.	2	S/S/S	11/8/170	78/78/767	115/115/20
$x+yz$, 4 sh.	3	I/I/I	11/7/8	102/102/72	99/99/15
$x+yz+xyz$, 4 sh.	1	S/S/S	45/18/175	31/31/167	63/63/1
xy virtual var.	1	S/S/S	3/1/17	9/9/55	23/23/0
xy virtual share	1	S/S/S	1/0/7	9/9/37	15/15/0
xy, 4→3 sh.	1	S/S/S	1/1/3	11/11/25	8/8/0
d+1 **quadratic S-box family** (Bilgin et al., 2015)					
Q_1^3	1	S/S/S	0/1/0	13/13/23	11/11/10

continued on next page

Table 7.1, continued

Circuit	d	Coupling / Replay / Closure			
		Verdict	Time [ms]	Discharges	Free wires
Q_2^3	1	S/I†/S	2/1/4	25/24/43	13/13/6
Q_3^3	1	I/I/I	4/2/7	18/18/32	22/22/2
Q_4^4	1	S/S/S	1/0/1	15/15/27	14/14/14
Q_{12}^4	1	S/I†/S	2/2/5	25/24/47	15/15/8
Q_{293}^4	1	S/I†/S	4/2/6	31/24/59	20/20/8
Q_{294}^4	1	S/S/S	1/1/3	23/23/39	16/16/12
Q_{299}^4	1	S/I†/S	5/2/13	35/18/75	26/22/8
Q_{300}^4	1	I/I/I	4/3/8	18/18/32	24/24/2

Three observations stand out. First, coupling matches replay's yield exactly. On every entry both engines verify, their discharge counts and free-wire yields agree to the digit, 4735 discharges and 8078 free wires for the five-share ISW multiplication on both for example. Replay is a small constant faster, having no factoring attempts to make, but it pays for that speed with the five false positives of the factoring family. Coupling attains replay's yield while keeping the correct verdict. Second, the extension pays for itself greatly. On the Fides S-box a single run makes only 55 engine discharges yet certifies 537 wires for free, on a gadget of some 900 wires, and against the naive baseline that discharges each probe alone for the five-share ISW multiplication falls from roughly 609 seconds to under two, a factor of about 337 (Figure 7.1). The free wires are the mechanism by which the Vandermonde split prunes, so a generous yield directly implies a smaller search. Third, closures keep the verdicts but fail on efficiency. Retaining factoring, the closure engine agrees with coupling on every verdict, but its local, topological deduction yields far fewer free wires as the traversal fragments, 14 against 537 on Fides and 645 against 2671 on the three-share DOM Keccak χ, so the split degrades toward enumeration: the five-share ISW multiplication needs 632,834 discharges under closures against 4,735 under coupling. The wall-clock gap is also considerably larger.

7.4 Discussion

The evaluation supports that on the standard masking gadgets Coppula is exact and fast, certifying the secure constructions and returning true witnesses on the insecure ones, and its coupling extension attains the yield of witness replay while keeping the verdict quality that factoring buys. The d+1 quadratic family is a concrete class of

circuits it certifies where a factoring-free, `maskVerif`-style engine does not.

The limits are also made clear. The model is value-based, so the glitch and transition leakage that a hardware verifier must eventually treat is out of scope (Chapter 8); the tool verifies a circuit whole and does not compose gadgets; and, as Figure 7.1 shows, verification time grows exponentially in a wire's multiplicative depth. Our contribution is not so much scale or performance but exactness through a constructive and coupling-based certificate, together with the family of low-randomness compression circuits that this certificate brings within reach.

8. FUTURE WORK

The tool as it stands leaves several research directions unaddressed. Some of these directions sharpen the engine without changing the model it verifies, others widen the model, making the hardware it protects represent a larger, more realistic class. We collect them here, from the most immediate to the most ambitious.

8.1 Arithmetic Circuits on Larger Fields

It is tempting to file work over larger fields under "not yet supported," but much of the machinery is already field-agnostic and it is worth saying exactly is supported and what is not. The coupling characterisation of Chapter 4 and the subset-space traversal of Section 5.3 never used any property of $\mathbb{F}_2$ beyond a finite randomness space sampled uniformly, so they carry over to any finite field trivially. The substitution at the heart of the engine generalises just as cleanly. Over $\mathbb{F}_q$ the invertible affine map $r \mapsto c \cdot r + e$ with $c \neq 0$ and $r \notin \mathrm{vars}(e)$ is again a bijection of the randomness space and plays exactly the role the shear $r \mapsto e \oplus r$ plays over $\mathbb{F}_2$.

What is genuinely tied to $\mathbb{F}_2$ is the concrete engine built on that substitution. The exact endpoint test when extending a probe set rests on the algebraic normal form, whose multilinearity is the $\mathbb{F}_2$ identity $x^2 = x$; the multiplicative factoring of Section 6.1.3 and the canonicalisation it runs through are likewise $\mathbb{F}_2$ algebra, as is the two-input XOR/AND gate basis. So the step to a larger field necessitates to build on top rather than to redesign. One can replace the endpoint with a suitable normal form, or with a Schwartz–Zippel probabilistic identity test (though that would harm the exactness), which is in fact cheaper in the larger field, re-derive factoring for the new algebra, and enlarge the gate set. Verification is already moving this

way, toward arithmetic and prime-field masking under the pressure of post-quantum schemes (Gigerl et al., 2023) which work on larger fields, and a field-generic version of our coupling engine would fit naturally into that line.

8.2 Factoring Heuristics

The multiplicative factoring of Section 6.1.3 pulls out, greedily, the factor shared by the most product terms, breaking ties by identifiers. Factoring is not confluent: which factor one extracts, and in what order relative to the substitutions, decides whether a buried random surfaces and how much cancellation follows. The present choice is cheap and suffices on our benchmarks, but it is surely not optimal. One might weight a candidate factor by whether pulling it out actually exposes an additive single-occurrence random rather than merely by how often it appears, i.e. it can explore a shallow lookahead over factor choices, or infer an order from the gadget's structure. Because factoring preserves a wire's value (Section 6.1.3.2), every such heuristic is sound by construction; the question is purely how often, and how quickly, it reaches a discharge. A systematic study of these heuristics, measured against the completeness they buy, is left open.

8.3 Composing Couplings for Unions of Safe Sets

Component III certifies one safe set from one coupling. When the traversal certifies probes on sibling branches it obtains two safe sets S_1 and S_2, each internally secret-independent, with couplings φ_1 and φ_2. Their union need not be secret-independent of course, and the present scheme makes no attempt on it; the branches are kept apart. The couplings themselves, though, may sometimes be composed into a certificate for the union.

Call the *randomness footprint* of a safe set the randoms its wires depend on together with the randoms its coupling relabels. Suppose S_1 and S_2 have disjoint footprints. Then φ_1 and φ_2 move disjoint coordinates of the randomness space, so they commute nicely, and neither disturbs the wires the other has already blinded. Their composite

$\varphi_1 \circ \varphi_2$ is again a bijection, and it carries every wire of $S_1 \cup S_2$ to a secret-free form. In such a case, a wire of S_1 is blinded by φ_1 and left untouched by φ_2, and symmetrically for S_2. By the safe set fact of Section 5.4.3 that single composite certifies the whole union, merging two safe sets with no fresh discharge.

The disjointness is the key here, and it is why the idea is a direction and not yet a method. When the footprints overlap, as two gadgets sharing a mask, the shears no longer commute and one coupling can undo the blinding the other needs, so the composite is not automatically a coupling for the union. Detecting the disjoint case cheaply, and finding a principled combination when footprints overlap, is the open problem. Its payoff would be direct. The traversal could fuse the safe sets of neighbouring cells and prune the subset space far more aggressively than certifying each in isolation allows.

8.4 Symmetry-Orbit Space Exploration

The subset-space traversal treats all wires alike, but masked gadgets are highly symmetric. Permuting the share indices of an input, or relabelling interchangeable masks, carries one probe to another with identical secret-dependence. Such a relabelling is a circuit automorphism, and the automorphisms partition the probes into orbits on which the verdict is constant, so verifying one representative per orbit would certify the entire orbit and shrink the space the traversal must explore by roughly the size of the symmetry group. Cutting a search by exploiting its symmetry this way is a known technique in model checking (Clarke et al., 1996). Carrying it into our setting, by detecting a gadget's automorphisms and quotienting the Vandermonde recursion of Section 5.3.3 by them, is a promising and, to our knowledge, largely unexplored direction for language-based probing-security verification.

8.5 Scaling and Compositional Verification

We have previously mentioned that probing security does not compose, so the approach does not, on its own, scale to a full cipher assembled from gadgets. The com-

posability notions surveyed in Section 3.3.2 are the established works in this line. Strong non-interference (Barthe et al., 2016) and probe-isolating non-interference (Cassiers & Standaert, 2020) let one certify small gadgets once and wire them together with the order preserved, while tight approaches (Belaïd et al., 2018) track exactly the refreshing this needs. Our engine could serve as the per-gadget oracle inside such a scheme, discharging each gadget's stronger non-interference obligation while a compositional layer handles the assembly. Adapting the work to include such compositional methods is work that remains to be explored.

8.6 Glitches and Practical Aspects

Our model is value-based (Chapter 2). So, a probed wire leaks its logical value and nothing more. Any hardware designer knows that reality is a lot less forgiving. A gate glitches on unsettled inputs and a register transition leaks a function of two successive values, and either can reopen a leak that a value-based check certifies. The glitch-extended probing model, formalised together with composition in the robust probing model (Faust et al., 2018), captures these effects, and some tools verify masking under them (Barthe et al., 2019; Bloem et al., 2018), with recent work spanning both Boolean and arithmetic masking across hardware and software (Gigerl et al., 2023). Lifting our checker to a glitch-extended model would mean charging a probe on a glitching gate with all of that gate's stable inputs, enlarging each observation before the engine ever sees it. The coupling machinery would carry over easily as well; what changes is the front end that forms the probes. Validating the result on real netlists is the practical work that would turn the tool from a checker of simple circuits into a verifier of practical implementations.

9. CONCLUSION

Masking is prone to design flaws. Shares of a secret might entangle as non-linearity necessitates it to a degree. Hence, a checker verifying that too many shares never come back together is of value. This thesis set out to make that check exact. The question we fixed on is d-probing security of masked arithmetic circuits over $\mathbb{F}_2$, checking whether every set of at most d wires is jointly independent of the secrets, and our aim throughout was to answer it without approximation, either certifying a circuit or handing back a concrete probe that we suspect to leak. The difficulty is that exactness is expensive, and much of the work was in finding a route that pays for it in an accaptable way.

We began by measuring the cost precisely. The law of a probe is a histogram of tape counts, and we showed that computing one such count is #P-complete while deciding whether two secrets leave the law unchanged is $\mathrm{C}_{=}$P-complete (Chapter 4). Even a single probe therefore lands above the polynomial hierarchy, which settles that no exact procedure built on mere counting can be efficient and points away from the distribution and toward its structure. This is the motivation that shapes our approach.

The rest of the thesis is the constructive answer, an executable checker, Coppula, organised around three components and one idea that ties them together. Heavily based on Barthe et al.'s work on language-based verification (Barthe et al., 2015, 2019). Component I, the rewrite engine, decides a single probe by substituting its randoms away one at a time until no secret remains, cheaply when each random occurs once and by a generalized fallback otherwise, with a multiplicative factoring step that reaches masks buried inside products (Chapter 5). Component II sweeps every d-subset without enumerating them, splitting the space recursively and losing nothing, which we proved by Vandermonde's identity at the level of sets. Component III grows a single certified probe into a whole safe set of wires, so that one discharge settles many probes at once. The idea underneath all three is the *coupling*.

A successful discharge is not merely a secure verdict, but a measure-preserving relabelling of the randomness under which the secrets visibly cancel, and reading it that way is what lets one certificate vouch for a probe, for its neighbour wires, and for the soundness of the traversal that calls it. The resulting checker is a sound but incomplete tool.

We implemented Coppula in Lean 4 on a hash-consed circuit representation with a bit-sliced test for the extension (Chapter 6), and evaluated it on a suite of masked gadgets (Chapter 7), where it certifies the secure constructions and, on an insecure one, returns a witnessing probe.

What we take from the work is that couplings give a benefical view on probing security. They turn a distributional statement into a constructive object one can build, record, and replay, and they unify the three components under a single soundness argument rather than three. As stated, the approach stands in the language-based lineage of Barthe et al. (Barthe et al., 2015, 2019), and its departures, the explicit coupling with an exact endpoint, the factoring, the bit-sliced fast-reject test, are what this thesis contributes to that line. Much is left open, from larger fields and glitches to composition and to the composing of couplings across safe sets (Chapter 8), and the hardness result guarantees that no single technique will close the gap completely.

BIBLIOGRAPHY

Barthe, G., Belaïd, S., Cassiers, G., Fouque, P.-A., Benjamin, G., & Standaert, F.-X. (2019). Maskverif: Automated verification of higher-order masking in presence of physical defaults. In K. Sako, S. A. Schneider, & P. Y. A. Ryan (Eds.), *Computer security – esorics 2019* (pp. 300–318, Vol. 11735). Springer International Publishing. https://doi.org/10.1007/978-3-030-29959-0_15

Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P.-A., Grégoire, B., & Strub, P.-Y. (2015). Verified proofs of higher-order masking. *Advances in Cryptology – EUROCRYPT 2015*, *9056*, 457–485. https://doi.org/10.1007/978-3-662-46800-5_18

Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P.-A., Grégoire, B., Strub, P.-Y., & Zucchini, R. (2016). Strong non-interference and type-directed higher-order masking. *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 116–129. https://doi.org/10.1145/2976749.2978427

Barthe, G., Grégoire, B., Hsu, J., & Strub, P.-Y. (2017). Coupling proofs are probabilistic product programs. *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 161–174. https://doi.org/10.1145/3009837.3009896

Belaïd, S., Goudarzi, D., & Rivain, M. (2018). Tight private circuits: Achieving probing security with the least refreshing. *Advances in Cryptology – ASIACRYPT 2018*, *11273*, 343–372. https://doi.org/10.1007/978-3-030-03329-3_12

Bilgin, B., Nikova, S., Nikov, V., Rijmen, V., Tokareva, N., & Vitkup, V. (2015). Threshold implementations of small s-boxes. *Cryptography and Communications*, *7*(1), 3–33. https://doi.org/10.1007/s12095-014-0104-7

Bloem, R., Groß, H., Iusupov, R., Könighofer, B., Mangard, S., & Winter, J. (2018). Formal verification of masked hardware implementations in the presence of glitches. *Advances in Cryptology – EUROCRYPT 2018*, *10821*, 321–353. https://doi.org/10.1007/978-3-319-78375-8_11

Cassiers, G., & Standaert, F.-X. (2020). Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Transactions on Information Forensics and Security*, *15*, 2542–2555. https://doi.org/10.1109/TIFS.2020.2971153

Chari, S., Jutla, C. S., Rao, J. R., & Rohatgi, P. (1999). Towards sound approaches to counteract power-analysis attacks. *Advances in Cryptology – CRYPTO '99*, *1666*, 398–412. https://doi.org/10.1007/3-540-48405-1_26

Clarke, E. M., Enders, R., Filkorn, T., & Jha, S. (1996). Exploiting symmetry in temporal logic model checking. *Formal Methods in System Design*, *9*(1–2), 77–104. https://doi.org/10.1007/BF00625969

Duc, A., Dziembowski, S., & Faust, S. (2014). Unifying leakage models: From probing attacks to noisy leakage. https://eprint.iacr.org/2014/079

Eldib, H., Wang, C., & Schaumont, P. (2014). Formal verification of software countermeasures against side-channel attacks. *ACM Transactions on Software Engineering and Methodology*, *24*(2), 11:1–11:24. https://doi.org/10.1145/2685616

Faust, S., Grosso, V., Merino Del Pozo, S., Paglialonga, C., & Standaert, F.-X. (2018). Composable masking schemes in the presence of physical defaults and the robust probing model. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, *2018*(3), 89–120. https://doi.org/10.13154/tches.v2018.i3.89-120

Fenner, S. A., Fortnow, L. J., & Kurtz, S. A. (1994). Gap-definable counting classes. *Journal of Computer and System Sciences*, *48*(1), 116–148. https://doi.org/10.1016/S0022-0000(05)80024-8

Gao, P., Zhang, J., Song, F., & Wang, C. (2019). Verifying and quantifying side-channel resistance of masked software implementations. *ACM Transactions on Software Engineering and Methodology*, *28*(3), 16:1–16:32. https://doi.org/10.1145/3330392

Gigerl, B., Primas, R., & Mangard, S. (2023). Formal verification of arithmetic masking in hardware and software. *Applied Cryptography and Network Security – ACNS 2023*, *13905*. https://doi.org/10.1007/978-3-031-33488-7_1

Groß, H., Mangard, S., & Korak, T. (2016). Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. *Proceedings of the 2016 ACM Workshop on Theory of Implementation Security (TIS@CCS)*, 3. https://doi.org/10.1145/2996366.2996426

Ishai, Y., Sahai, A., & Wagner, D. (2003). Private circuits: Securing hardware against probing attacks. In D. Boneh (Ed.), *Advances in cryptology - crypto 2003* (pp. 463–481). Springer Berlin Heidelberg.

Knichel, D., Sasdrich, P., & Moradi, A. (2020). SILVER – statistical independence and leakage verification. *Advances in Cryptology – ASIACRYPT 2020*, *12491*, 787–816. https://doi.org/10.1007/978-3-030-64837-4_26

Kocher, P., Jaffe, J., & Jun, B. (1999). Differential power analysis. *Advances in Cryptology – CRYPTO '99*, *1666*, 388–397. https://doi.org/10.1007/3-540-48405-1_25

Nikova, S., Rechberger, C., & Rijmen, V. (2006). Threshold implementations against side-channel attacks and glitches. *Information and Communications Security – ICICS 2006*, *4307*, 529–545. https://doi.org/10.1007/11935308_38

Rivain, M., & Prouff, E. (2010). Provably secure higher-order masking of AES. *Cryptographic Hardware and Embedded Systems – CHES 2010*, *6225*, 413–427. https://doi.org/10.1007/978-3-642-15031-9_28

Toda, S. (1991). PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, *20*(5), 865–877. https://doi.org/10.1137/0220053

Trichina, E. (2003). Combinational logic design for AES SubByte transformation on masked data. https://eprint.iacr.org/2003/236

Valiant, L. G. (1979). The complexity of computing the permanent. *Theoretical Computer Science*, *8*(2), 189–201. https://doi.org/10.1016/0304-3975(79)90044-6

Wagner, K. W. (1986). The complexity of combinatorial problems with succinct input representation. *Acta Informatica*, *23*(3), 325–356. https://doi.org/10.1007/BF00289117

Xiao, G.-Z., & Massey, J. L. (1988). A spectral characterization of correlation-immune combining functions. *IEEE Transactions on Information Theory*, *34*(3), 569–571. https://doi.org/10.1109/18.6037

APPENDIX - ADDITIONAL EXPERIMENTAL DATA

This appendix transcribes two representative gadgets from the evaluation suite of Chapter 7 as $\mathbb{F}_2$ netlists, in the same form the tool consumes them. Input shares are listed first and are not probe targets. Every remaining wire is a probe target. Addition is $\oplus$ and multiplication is $\odot$.

DOM-AND (Two Shares)

The following is the domain-oriented masking of a single AND (Groß et al., 2016), secure at order one. The secrets are shared as $a = a_0 \oplus a_1$ and $b = b_0 \oplus b_1$, with a_0, b_0 as fresh randoms, and r as fresh mask:

$$\begin{aligned} u_0 &= a_0 \odot b_0, & u_1 &= a_1 \odot b_1, \\ p_{01} &= a_0 \odot b_1, & p_{10} &= a_1 \odot b_0, \\ c_{01} &= p_{01} \oplus r, & c_{10} &= p_{10} \oplus r, \\ s_0 &= u_0 \oplus c_{01}, & s_1 &= u_1 \oplus c_{10}. \end{aligned}$$

The outputs s_0, s_1 re-share $a \odot b$, and the two masked cross-terms c_{01}, c_{10} carry the same fresh mask r, which makes each of them individually uniform. The mask is shared across the pair by design, so it cancels in $c_{01} \oplus c_{10} = p_{01} \oplus p_{10} = a_0 \odot b \oplus a \odot b_0$, which is a secret-dependent value. A joint probe on the pair therefore leaks, which is why this two-share DOM-AND is secure at order one but not at order two (Table 7.1).

$X + YZ$ Threshold Implementation (Four Shares)

The four-share TI of $x \oplus (y \odot z)$ (Bilgin et al., 2015), the order-three case study of Section 7.3.1. Each input is four-shared, $x = x_1 \oplus x_2 \oplus x_3 \oplus x_4$ and likewise for y, z, with x_1, x_2, x_3 fresh randoms and $x_4 = x \oplus x_1 \oplus x_2 \oplus x_3$. All sixteen cross-products

$y_i \odot z_j$ are formed, and the four output shares are

$$
\begin{aligned}
F_1 &= x_2 \oplus \bigoplus_{i,j \in \{2,3,4\}} y_i z_j, \\
F_2 &= x_3 \oplus y_1 z_3 \oplus y_1 z_4 \oplus y_3 z_1 \oplus y_4 z_1 \oplus y_1 z_1, \\
F_3 &= x_4 \oplus y_1 z_2 \oplus y_2 z_1, \\
F_4 &= x_1.
\end{aligned}
$$

The sharing computes $x \oplus (y \odot z)$. It is a first-order threshold implementation, certified secure at orders one and two and insecure at order three (Chapter 7).